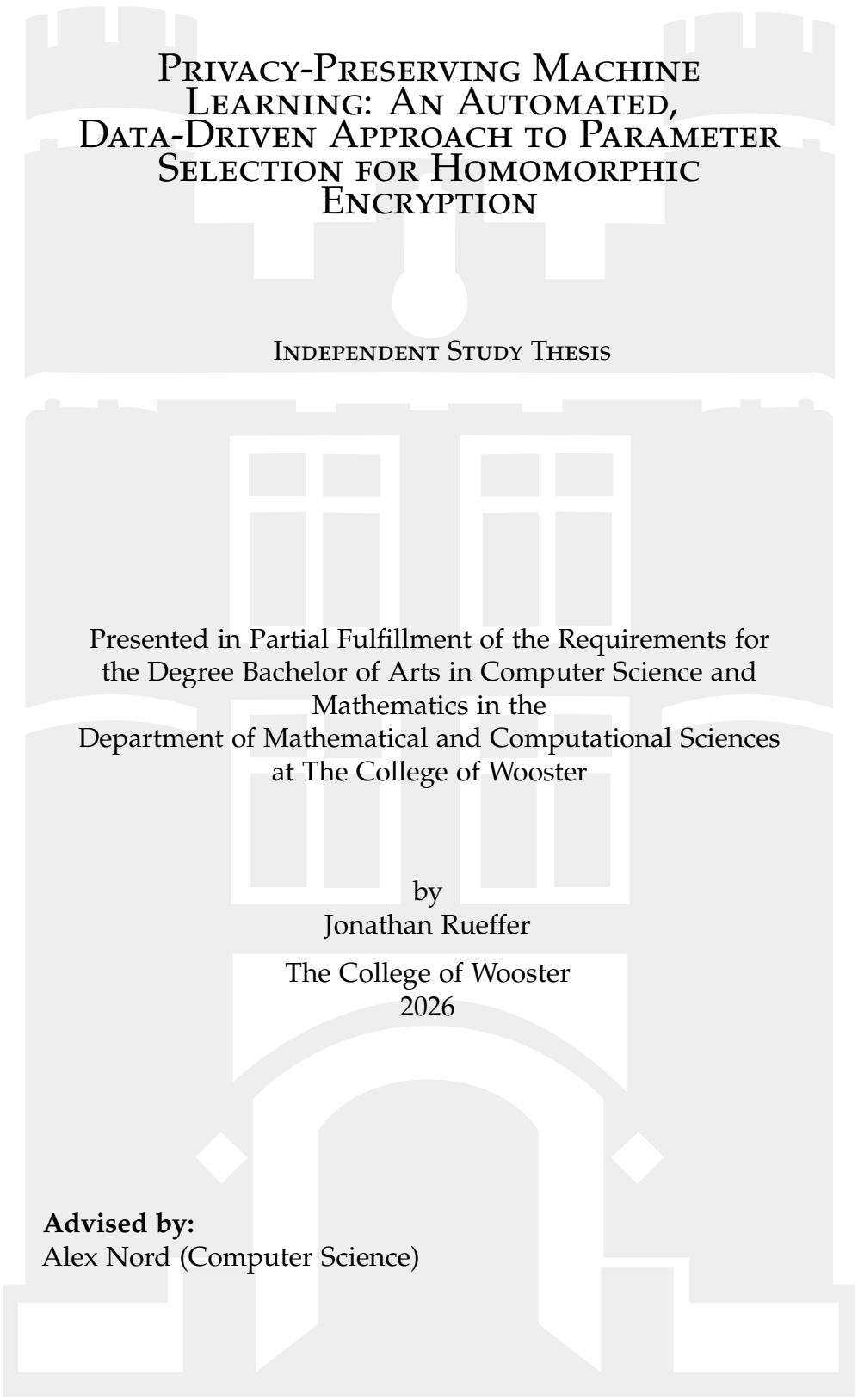




PRIVACY-PRESERVING MACHINE LEARNING: AN AUTOMATED, DATA-DRIVEN APPROACH TO PARAMETER SELECTION FOR HOMOMORPHIC ENCRYPTION

INDEPENDENT STUDY THESIS

Presented in Partial Fulfillment of the Requirements for
the Degree Bachelor of Arts in Computer Science and
Mathematics in the
Department of Mathematical and Computational Sciences
at The College of Wooster

by
Jonathan Rueffer

The College of Wooster
2026

Advised by:
Alex Nord (Computer Science)



THE COLLEGE OF
WOOSTER

© 2026 by Jonathan Rueffer
All rights reserved.

ABSTRACT

Homomorphic encryption enables computation directly on encrypted data, offering a promising solution for outsourced machine learning in privacy-critical settings, such as fraud detection in banking and medical diagnosis in healthcare. In particular, the Cheon–Kim–Kim–Song (CKKS) scheme is well-suited for machine learning inference, as it supports arithmetic over real-valued data. However, its practical deployment remains challenging due to the complexity of parameter selection, which requires balancing security, numerical precision, and computational efficiency. This thesis presents an automated, data-driven framework for CKKS parameter selection that incorporates both dataset characteristics and model structure to guide the generation and evaluation of candidate parameter configurations. Central to this approach is the use of test vectors representative of the underlying data, without requiring direct access to sensitive information. The parameter configurations are analyzed through a multi-objective trade-off framework, allowing users to reason about various performance metrics rather than relying on a single recommended solution. Experimental results demonstrate that the proposed framework identifies parameter configurations with optimal trade-offs while significantly reducing the computational cost of an exhaustive parameter search, and that representative test vectors effectively approximate real inference behavior. Overall, this work contributes to improving the usability of privacy-preserving machine learning by enabling automated, data-driven parameter selection while supporting exploration of application-specific trade-offs.

ACKNOWLEDGMENTS

I would like to thank Prof. Alex Nord for his guidance throughout the development and writing of this thesis. I would also like to sincerely thank my family and friends for their continued encouragement and support.

CONTENTS

Abstract	v
Acknowledgments	vii
Contents	ix
List of Figures	xii
List of Tables	xiii

CHAPTER	PAGE
1 Introduction	2
2 Background	4
2.1 Homomorphic Encryption	4
2.1.1 Definition of Homomorphic Encryption	4
2.1.2 Arithmetic Circuits as a Model of Computation	5
2.1.3 Classification of Homomorphic Encryption Schemes	6
2.1.3.1 Classification by Computational Capability	6
2.1.3.2 Classification by Arithmetic Capability	8
2.1.3.2.1 Exact Arithmetic Schemes	8
2.1.3.2.2 Approximate Arithmetic Schemes	9
2.1.4 Applications of Homomorphic Encryption	9
2.1.5 Public-Key Homomorphic Encryption Framework	11
2.1.5.1 Structure of a Public-Key Encryption Scheme	11
2.1.5.2 Correctness	12
2.1.5.3 Security	13
2.2 Hardness Foundation of Homomorphic Encryption	15
2.2.1 The Learning With Errors (LWE) Problem	15
2.2.2 Ring Learning With Errors (RLWE)	17
2.2.3 RLWE-based Encryption Schemes	17
2.3 The CKKS Scheme	18
2.3.1 Encoding	20
2.3.1.1 Vanilla Encoding	20
2.3.1.2 CKKS Encoding	22
2.3.2 Encryption and Decryption	24
2.3.2.1 Relinearization	28

2.3.2.2	Rescaling	30
2.3.3	CKKS Implementations	32
2.3.3.1	Practical Optimizations	33
2.3.3.2	Security	34
2.3.3.3	CKKS Libraries	35
2.3.4	CKKS Parameters	37
3	Related Work	40
3.1	Parameter Selection in CKKS Libraries	40
3.2	Compiler-Based Systems for Encrypted Machine Learning	41
3.3	Application-Aware Security and Parameterization	42
3.4	Automated Parameter Selection Research	43
3.5	Practical Guidelines for CKKS Parameter Selection	44
3.6	Summary and Motivation	44
4	System Design	46
4.1	Input Processing	49
4.1.1	Data-Driven Precision Estimation	49
4.1.1.1	Integer Precision Estimation	50
4.1.1.2	Decimal Precision Estimation	50
4.1.2	Model Compatibility Constraints	51
4.1.3	Simulation of Client–Server Deployment	52
4.1.4	Built-in Datasets and Model Construction	52
4.2	Test Vector Construction	53
4.3	Search Space Generation	54
4.3.0.1	Selecting Candidate Polynomial Degrees	55
4.3.1	Generating Candidate Scales	56
4.3.2	Constructing Candidate Modulus Chains	57
4.3.2.1	Choosing the First Modulus q_0	57
4.3.2.2	Choosing the Chain Length L	57
4.3.2.3	Choosing the Final Modulus q_L	58
4.3.2.4	Constructing the Full Modulus Chain	58
4.3.3	Final Search Space	58
4.4	Parameter Evaluation	59
4.4.1	Empirical Evaluation Metrics	59
4.4.1.1	Precision	60
4.4.1.2	Runtime	60
4.4.2	Analytical Evaluation Metrics	61
4.4.2.1	Security	61
4.4.2.2	Memory	62
4.4.3	Final Evaluation Vector	63
4.5	Parameter Recommendation	64
4.5.1	Pareto Frontier Construction	64
4.5.2	Selecting the Knee Point	65
4.5.3	Final Inference Evaluation	66
4.6	Output	67

4.6.1	Terminal Output	68
4.6.2	File Output	68
5	Evaluation	70
5.1	Experimental Setup	71
5.1.1	Datasets	71
5.1.2	Models	71
5.1.2.1	Synthetic Polynomial Model	72
5.1.2.2	Dataset-Trained Models	72
5.2	Runtime Efficiency	72
5.3	Test Vector Generation Strategy	74
5.3.1	Sampling Strategy	74
5.3.2	Number of Test Vectors	76
5.3.3	Comparison with Random Dataset Sampling	77
5.4	Parameter Search Space Efficiency	78
5.5	CKKS Parameter Tradeoffs	81
5.5.1	Runtime and Multiplicative Depth	81
5.5.2	Precision and Multiplicative Depth	82
5.6	Case Studies	83
5.6.1	Diabetes Dataset	84
5.6.2	Wisconsin Breast Cancer Dataset	87
6	Discussion	90
6.1	Data-Driven Parameter Selection	91
6.2	Trade-off Analysis and Usability	92
6.3	Workload-Specific Parameter Exploration	93
6.4	Future Work	93
6.4.1	Dataset and Model Scope	93
6.4.2	Representative Test Vector Security	94
7	Conclusion	96
	References	98
APPENDIX		PAGE
A	Generative AI Usage Statement	104

LIST OF FIGURES

Figure	Page
2.1 Schematic overview of the CKKS encryption scheme.	19
2.2 Computational pipeline for homomorphic multiplication in CKKS.	32
2.3 Dependency structure between CKKS parameters and application requirements.	39
4.1 Six-stage pipeline for CKKS parameter selection.	48
4.2 Illustration of a Pareto frontier showing dominated, non-dominated, and knee points.	66
5.1 Tool runtime as a function of multiplicative depth.	73
5.2 Accuracy of test vector sampling strategies	75
5.3 Accuracy comparison of test vector sampling strategies.	77
5.4 Error convergence of test vector generation strategies.	78
5.5 Comparison of full parameter search space and tool-based exploration. . .	80
5.6 Encrypted inference runtime as a function of multiplicative depth (by security level).	82
5.7 Encrypted inference precision as a function of multiplicative depth (by security level).	83
5.8 Precision–runtime tradeoff at 128-bit security level (Diabetes dataset). . . .	85
5.9 Precision–runtime tradeoff at 192-bit security level (Diabetes dataset). . . .	85
5.10 Precision–runtime tradeoff at 256-bit security level (Diabetes dataset). . . .	86
5.11 Precision–runtime tradeoff of all security levels (Diabetes dataset).	86
5.12 Precision–runtime tradeoff at 128-bit security level (Breast Cancer dataset). .	87
5.13 Precision–runtime tradeoff at 192-bit security level (Breast Cancer dataset). .	88
5.14 Precision–runtime tradeoff at 256-bit security level (Breast Cancer dataset). .	88
5.15 Precision–runtime tradeoff of all security levels (Breast Cancer dataset). . .	89

LIST OF TABLES

Table	Page
2.1 Description of data at each stage of the CKKS pipeline.	19
2.2 Comparison of CKKS implementations across popular FHE libraries.	37
2.3 Dependency relationships between CKKS parameters and application requirements.	39
4.1 Description of the six-stage CKKS parameter selection pipeline.	48
4.2 Maximum multiplicative depth for each polynomial modulus degree.	55
4.3 Evaluation metrics used in the parameter recommendation framework.	59
4.4 Maximum modulus chain sizes for 128-bit security.	61
4.5 Maximum modulus chain sizes for 192-bit security.	62
4.6 Maximum modulus chain sizes for 256-bit security.	62
4.7 Estimated memory requirements of CKKS components.	63
4.8 Description of output CSV file columns.	69
5.1 Comparison of runtime and candidate parameter counts between tool-based and near-exhaustive search.	79

CHAPTER 1

INTRODUCTION

Modern machine learning models offer powerful predictive capabilities across domains such as finance and healthcare, where they can be used to expose fraud and assist in medical diagnosis [31]. However, these models are often developed and hosted by third-party providers, requiring users to submit sensitive data to external platforms for inference. For example, a hospital seeking to use a cloud-based diagnostic model must send patient data to a remote server, where the model processes the input and returns a prediction to the client.

This standard Software as a Service deployment model introduces significant privacy concerns [31]. Sensitive data, such as financial records or patient medical information, is strictly regulated, including the General Data Protection Regulation (GDPR) in the European Union and the Health Insurance Portability and Accountability Act (HIPAA) in the United States. Therefore, directly sharing raw data with external services is often impractical or prohibited, limiting the adoption of powerful machine learning tools in privacy-critical domains.

Homomorphic encryption offers a compelling solution to this problem [19], as it is a form of encryption that allows computations to be performed directly on encrypted data without decrypting it first. Under this paradigm, a client can encrypt their input data, such as a patient’s medical record, send it to a remote model, and receive an encrypted result that can be decrypted locally to obtain the correct output, i.e., as if the model had

used the raw data. In particular, the Cheon–Kim–Kim–Song (CKKS) [9] homomorphic encryption scheme is used in such machine learning applications due to its support of computations involving real numbers.

Despite its theoretical promise, CKKS remains challenging to deploy in practice [35]. A major obstacle is the complexity of its parameterization, which directly affects security, computational cost, and numerical precision. Selecting appropriate parameters requires carefully balancing these competing objectives, and existing approaches often rely on manual tuning or broad guidelines. This creates a significant barrier to entry, particularly for non-experts in homomorphic encryption.

This thesis addresses this challenge by proposing a data-driven parameter recommendation framework for CKKS-based machine learning inference. The proposed tool automates the parameter selection process by incorporating both dataset characteristics and model structure into the selection of candidate configurations. In particular, these candidate configurations are evaluated using a proposed representative test vector strategy to approximate inference error without requiring direct use of sensitive data. The tool presents parameter recommendations as a trade-off analysis across runtime, precision, and security.

By automating and framing parameter selection as a structured, data-informed process, this work aims to improve the usability of CKKS-based privacy-preserving machine learning systems. The resulting framework enables users to make informed decisions about parameter configurations without relying on user expertise or an exhaustive search.

This thesis is organized as follows. Chapter 2 provides the necessary background on homomorphic encryption and the CKKS scheme. Chapter 3 reviews existing approaches to parameter selection. Chapter 4 presents the design of the proposed framework, and Chapter 5 evaluates its performance across several experimental settings. Finally, Chapter 6 discusses the broader implications of these results.

CHAPTER 2

BACKGROUND

This chapter introduces the foundations of homomorphic encryption, the mathematical hardness assumptions that underpin its security, and a formal definition of the CKKS scheme. These concepts form the basis for the parameter selection framework developed in this thesis.

2.1 HOMOMORPHIC ENCRYPTION

Homomorphic encryption (HE) enables computation directly on encrypted data by preserving the underlying algebraic structure. This section provides its formal definition, surveys the different classifications and capabilities of HE schemes, and introduces the public-key HE framework used in privacy-preserving machine learning.

2.1.1 DEFINITION OF HOMOMORPHIC ENCRYPTION

The concept of an encryption scheme that preserves algebraic structure under computation was first formalized by Rivest, Adleman, and Dertouzos in their 1978 paper “On Data Banks and Privacy Homomorphisms” [29]. A homomorphic encryption (HE) scheme enables certain computations to be performed directly on the ciphertexts such that, after decryption, the result matches the outcome of performing the same computation on the underlying plaintexts.

Let Enc and Dec denote the encryption and decryption algorithms, respectively. An encryption scheme is homomorphic with respect to a plaintext operation $*$ if for any two plaintexts a and b ,

$$\text{Dec}(\text{Enc}(a) * \text{Enc}(b)) = a * b.$$

In other words, HE encryption bypasses the need to decrypt ciphertext to perform computations, thus allowing data manipulation entirely within the encryption space. Crucially, this implies that, within the context of public-key encryption, the secret key is not needed to perform computations on encrypted data.

A definitive example is the widely-adopted RSA encryption scheme, which is homomorphic with respect to the multiplication operation. Under RSA, the encryption function for a plaintext message m is defined as

$$\text{Enc}(m) = m^e \pmod{N},$$

where e is the public encryption exponent and N is a product of two large primes p and q . For two messages m_1 and m_2 , we observe that

$$\text{Enc}(m_1)\text{Enc}(m_2) = m_1^e \pmod{N} m_2^e \pmod{N} = m_1^e m_2^e \pmod{N} = \text{enc}(m_1 m_2).$$

Thus, multiplication in the ciphertext domain corresponds to multiplication of plaintexts. However, RSA does not support homomorphic addition, which motivated researchers to develop encryption schemes that are homomorphic to multiple operations.

2.1.2 ARITHMETIC CIRCUITS AS A MODEL OF COMPUTATION

Arithmetic circuits provide a formal model for representing polynomial computations. An arithmetic circuit C over a field $\mathbb{F}$, with a set of input variables $x_1, \dots, x_n$, is defined as a directed acyclic graph (DAG).

Nodes in C with indegree zero are called input nodes and are labeled with either a variable x_i or a constant from $\mathbb{F}$. Every other node are gates, each labeled by an arithmetic operations, typically addition or multiplication. Additionally, the circuit has one or more designated output nodes.

A circuit computes a polynomial iteratively. An input node computes the polynomial it is labeled by, either x_i or $c \in \mathbb{F}$. An addition gate computes the sum of the polynomials computed by its input nodes. A multiplication gate computes the product of the polynomials computed by its input nodes.

The graphical nature of arithmetic circuits admits two natural complexity measures. The size of a circuit is the total number of gates it contains. The depth of a circuit is the length of the longest directed path from an input node to an output node. In the context of homomorphic encryption, a crucial property is the multiplicative depth of a circuit, which is defined as the maximum number of multiplication gates encountered along any such path.

Multiplication operations increase ciphertext noise significantly more than additions, consequently making the multiplicative depth of a circuit determine whether a given homomorphic encryption scheme can evaluate it correctly.

2.1.3 CLASSIFICATION OF HOMOMORPHIC ENCRYPTION SCHEMES

Homomorphic encryption schemes can be categorized from multiple complementary perspectives that capture different aspects of their computational behavior. This subsection introduces two standard classifications: one based on computational capability using arithmetic circuits, and another based on the type of arithmetic supported.

2.1.3.1 CLASSIFICATION BY COMPUTATIONAL CAPABILITY

Using arithmetic circuits as the mathematical foundation, homomorphic encryption schemes are commonly categorized according to the class of circuits they can evaluate.

Partial homomorphic encryption (PHE) supports the homomorphic evaluation of circuits that consist of only one type of gate: either addition or multiplication. For example, the RSA encryption scheme falls under this category.

Somewhat homomorphic encryption (SHE) supports both addition and multiplication, but only for circuits of limited complexity. In practice, these are often restricted to low-degree polynomials before accumulated noise renders decryption incorrect.

Leveled homomorphic encryption (LHE) supports the evaluation of arbitrary circuits composed of addition and multiplication gates up to a predetermined depth bound. The maximum depth is fixed at key-generation time and determines the largest circuit that can be evaluated without additional noise-management techniques.

Fully homomorphic encryption (FHE) supports the evaluation of arbitrary circuits comprised of both multiplication and addition gates with unbounded depth, and it is therefore the strongest notion of homomorphic encryption. For any circuit C and input x , an FHE scheme satisfies

$$\text{Dec}(\text{Eval}(C, \text{Enc}(x))) = C(x),$$

where Eval denotes homomorphic evaluation of the circuit.

For decades, constructing a fully homomorphic encryption scheme supporting both addition and multiplication was considered the holy grail of cryptography. This question remained open for more than 30 years until 2009, when Graig Gentry proposed the first FHE scheme in his PhD thesis. His original construction was designed using ideal lattices, and although current schemes employ other mathematical concepts, it is the foundational blueprint on which modern schemes are built [22]. It took many more years for researchers to develop schemes that were actually practical to implement, but Gentry's initial idea still serves as the foundational blueprint.

Gentry later introduced in [15] a technique called bootstrapping to convert a SHE scheme into a FHE scheme. The bootstrapping procedure reduces the error accumulated during homomorphic operations, allowing for the successful evaluation of an arbitrary circuit. It accomplishes this by re-encrypting a ciphertext to refresh its noise, restoring the capacity for further homomorphic evaluation while guaranteeing the same underlying plaintext.

2.1.3.2 CLASSIFICATION BY ARITHMETIC CAPABILITY

Homomorphic encryption schemes can also be classified according to the type of arithmetic they support. The distinction lies between schemes that preserve exact modular arithmetic and those that enable approximate real or complex arithmetic. Importantly, this separate capability is independent of whether a scheme is partially or fully homomorphic.

2.1.3.2.1 Exact Arithmetic Schemes Exact homomorphic encryption schemes perform integer additions and multiplications on encrypted data while maintaining exact precision. All operations in the ciphertext domain are modular and performed over a polynomial ring, usually $\mathbb{Z}_q[x]$. The “exact arithmetic” designation ensures that decryption recovers the mathematically identical result of the evaluated arithmetic circuit. This decryption property is guaranteed provided that the noise level is managed correctly to remain below a correctness bound, which is usually accomplished using a bootstrapping technique.

For an exact HE scheme with associated operations Enc , Dec , Eval , and a function f , then for a plaintext message m :

$$\text{Dec}(\text{Eval}(f, \text{Enc}(m))) = f(m).$$

The most common exact schemes are the BFV scheme [14] and the BGV scheme [6], which were introduced in 2012. These constructions are well-suited for applications requiring exact integer results, such as financial data analysis, database queries, or any

secure, precise computation. Similarly to most modern HE schemes, their security is based on the Ring Learning With Errors (RLWE) hardness assumption.

2.1.3.2.2 Approximate Arithmetic Schemes Approximate homomorphic encryption schemes are designed to perform computations on encrypted data while allowing for small, controlled approximation errors in the results. They support efficient computation over real or complex numbers by encoding them into polynomial rings, typically $\mathbb{Z}_q[x]$. Decrypting the result is close to the true value, but may differ slightly (e.g., decrypting the sum of encrypted 1 and 2 might yield 3.001 instead of 3).

For an approximate HE scheme with operations Enc , Dec , Eval , and a function f , then for a plaintext message m :

$$\text{Dec}(\text{Eval}(f, \text{Enc}(m))) = f(m) + \varepsilon,$$

for some small and bounded ε .

The most prominent example is the CKKS scheme [9] introduced in 2017, which enables approximate arithmetic through a rescaling procedure to manage the noise and prevent it from growing too large during homomorphic operations. Unlike exact HE schemes, CKKS does not work with integers and does not guarantee bitwise correctness of the decrypted result. Instead, it is designed for real numbers and floating-point arithmetic, and its approximation accuracy depends on parameter selection and noise growth. Approximate schemes are ideal for machine learning, signal processing, and data mining on encrypted data, where small numerical errors are acceptable.

2.1.4 APPLICATIONS OF HOMOMORPHIC ENCRYPTION

Homomorphic encryption enables computation on encrypted data without requiring decryption, thereby addressing a fundamental challenge in modern data-driven systems: how to outsource computation while preserving confidentiality. This capability is

particularly relevant in settings where sensitive data is processed by an untrusted third party, such as cloud service providers.

One prominent application is privacy-preserving machine learning. Modern artificial intelligence systems rely on large-scale datasets and computationally intensive training and inference procedures. However, data owners are often hesitant or legally unable to disclose raw data due to privacy regulations or competitive concerns. Homomorphic encryption allows data to remain encrypted throughout model training or inference, enabling collaborative or outsourced computation without revealing underlying plaintext data [36].

Neural networks and other machine learning models can be represented as arithmetic circuits composed primarily of additions and multiplications. Since fully homomorphic encryption schemes support the evaluation of such circuits over encrypted inputs, they provide a natural cryptographic foundation for secure inference. Additionally, the intensive computational cost of a FHE scheme makes training a model completely within the encrypted domain as mostly unviable.

Beyond machine learning, homomorphic encryption has applications in blockchain systems and smart contracts. In conventional blockchain architectures, transaction data and contract parameters are publicly visible to ensure verifiability. However, this transparency poses risks in applications such as electronic voting, auctions, and healthcare data management. By encrypting inputs prior to submission and performing contract evaluation over ciphertexts, homomorphic encryption enables successful privacy-preserving smart contract execution [36].

The ability to efficiently evaluate real-valued arithmetic circuits under encryption is central to many practical deployments of homomorphic encryption. The CKKS scheme is specifically constructed to enable such real-valued homomorphic computations.

2.1.5 PUBLIC-KEY HOMOMORPHIC ENCRYPTION FRAMEWORK

Privacy-preserving machine learning systems are built on top of a public-key encryption framework that defines the basic structure and procedures of key generation, encryption, and decryption. This subsection introduces this framework and then formalizes the notions of correctness and security in the context of homomorphic encryption.

2.1.5.1 STRUCTURE OF A PUBLIC-KEY ENCRYPTION SCHEME

Public-key cryptography is a subset of cryptographic systems that use pairs of mathematically related keys: a public key and a corresponding private key. The public key may be distributed openly, while security depends on keeping the private key secret.

In a public-key encryption system, any party with the public key can encrypt a message to get a ciphertext, but only those with the associated private key can decrypt the ciphertext to obtain the original message.

A public key homomorphic encryption scheme $\mathcal{E}$ is formally defined as a set of probabilistic polynomial-time (PPT) algorithms [22]:

$$(\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval}).$$

- **Key Generation**

$$(pk, sk, evk) \leftarrow \text{KeyGen}(1^\lambda),$$

where λ is the security parameter. It outputs the secret key sk , the public key pk , and the public evaluation key evk , which is needed to perform homomorphic operations over ciphertexts.

- **Encryption**

$$c \leftarrow \text{Enc}_{pk}(m),$$

where pk is the public key and m is the plaintext message. It outputs the ciphertext c .

- **Decryption**

$$m \leftarrow \text{Dec}_{sk}(c),$$

where sk is the secret key and c is the ciphertext. It outputs the recovered original plaintext message m provided that the ciphertext has not accumulated excessive noise.

- **Evaluation**

$$c_f \leftarrow \text{Eval}_{evk}(f, (c_1, \dots, c_t)),$$

where each c_i encrypts a plaintext m_i . The output ciphertext c_f satisfies the correctness condition:

$$\text{Dec}_{sk}(c_f) = f(m_1, \dots, m_t),$$

provided that the evaluated function f lies within the class of functions supported by the scheme.

A fundamental characteristic of a homomorphic encryption scheme $\mathcal{E}$ is the class of functions that $\mathcal{E}$ is able to evaluate. In practice, this is determined by the scheme's management of noise growth. Since homomorphic multiplication operations increase noise more than additions, the multiplicative depth of the arithmetic circuit representing f is the primary factor limiting correct evaluation.

2.1.5.2 CORRECTNESS

The correctness of an encryption scheme is a fundamental property ensuring that decryption successfully reverses encryption for authorized users. Since public-key encryption is nondeterministic, correctness is defined with respect to the internal randomness of the algorithms.

A general public-key encryption scheme is perfectly correct if for all security parameters λ , all key pairs $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$, and all messages in the message space $m \in \mathcal{M}$,

$$\Pr [\text{Dec}_{sk}(\text{Enc}_{pk}(m)) = m] = 1,$$

where the probability is taken over the randomness of KeyGen and Enc.

A public-key homomorphic encryption scheme is statistically correct for a function class $\mathcal{F}$ if for all $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$, all $f \in \mathcal{F}$, and all messages $m \in \mathcal{M}$,

$$\Pr [\text{Dec}_{sk}(\text{Eval}_{evk}(f, \text{Enc}_{pk}(m))) = f(m)] = 1 - \text{negl}(\lambda),$$

where the probability is taken over the randomness of the scheme's associated processes [24].

Thus, correctness holds as long as the noise accumulated during homomorphic evaluations of functions remains below a scheme-dependent threshold.

2.1.5.3 SECURITY

The primary security objective of a public-key homomorphic encryption (HE) scheme is to ensure the confidentiality of plaintext data in settings where computation is outsourced to an untrusted party. In this model, an adversary may observe ciphertexts and perform arbitrary supported homomorphic evaluations over them, but must not learn any information about the underlying plaintexts. Security is critically dependent on keeping the private key secret to ensure that only information is revealed by legitimate decryption [4].

The standard security notion against passive adversaries for public-key encryption is indistinguishability under chosen-plaintext attack (IND-CPA), which is equivalently known as semantic security [24]. A scheme is IND-CPA secure if no probabilistic

polynomial-time adversary, given the public key and access to encryptions of chosen messages, can distinguish between the encryptions of two messages with non-negligible advantage over random guessing. Formally, this property is presented as a game.

Let an adversary $\mathcal{A}$ select a pair of messages, $(m_0, m_1) \leftarrow \mathcal{A}(pk)$, and let b be a random chosen bit, $b \leftarrow \{0, 1\}$. The adversary then receives the ciphertext $c \leftarrow \text{Enc}_{pk}(m_b)$ and outputs a guess $b' \in \{0, 1, \perp\}$. The advantage of $\mathcal{A}$ is then defined as

$$\text{Adv}_{\mathcal{A}} = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

IND-CPA security requires

$$\text{Adv}_{\mathcal{A}} \leq \text{negl}(\lambda),$$

which ensures that the adversary's advantage must shrink as the security of the scheme grows.

IND-CPA security guarantees that ciphertexts and evaluations of ciphertexts reveal no partial information about their corresponding plaintexts.

Additionally, to maintain the IND-CPA property, evaluated ciphertexts must also remain indistinguishable. In particular, for a message $m \in \mathcal{M}$ and corresponding ciphertext $c = \text{Enc}_{pk}(m)$, and any computable function $f \in \mathcal{F}$, the evaluated ciphertext

$$c_f = \text{Eval}_{evk}(f, c)$$

must remain computationally indistinguishable from an encryption of $f(m)$. Thus, homomorphic evaluation must not introduce additional information leakage.

Modern homomorphic encryption schemes are typically proven IND-CPA secure via reductions to well-studied computational hardness assumptions. In particular, security is often based on the assumed hardness of the Learning With Errors (LWE) or Ring Learning With Errors (RLWE) problems. Under these assumptions, breaking the semantic security

of the public-key HE scheme would imply an efficient solution to the underlying lattice problem.

Finally, it is important to note that ciphertext noise management impacts scheme correctness, not the semantic security guarantees of the scheme, which come from computational hardness assumptions.

2.2 HARDNESS FOUNDATION OF HOMOMORPHIC ENCRYPTION

The security of modern encryption schemes is based on certain mathematical problems that are believed to be computationally intractable. These hardness assumptions ensure that recovering the underlying plaintext without the secret key remains practically impossible. This section introduces these foundations, beginning with the Learning With Errors (LWE) problem, extending to its ring-based variant (RLWE), and describing how these problems form the basis of practical homomorphic encryption schemes.

2.2.1 THE LEARNING WITH ERRORS (LWE) PROBLEM

The security of most modern homomorphic encryption schemes is based on the hardness of lattice problems, in particular the Learning With Errors (LWE) problem introduced by Regev in 2005 [28]. LWE can be viewed as the problem of solving noisy linear equations over a finite field.

Let $n \geq 1$ be a dimension parameter, $q \geq 2$ a modulus, and χ a probability distribution over $\mathbb{Z}_q$, typically a narrow Gaussian distribution. A secret vector $s \in \mathbb{Z}_q^n$ chosen uniformly at random. A LWE sample is generated by choosing $a \in \mathbb{Z}_q^n$ at random, sampling an error term $e \leftarrow \chi$. The output is the pair

$$(a, b) = (a, \langle a, s \rangle + e \bmod q).$$

The LWE problem is, given a sequence of independent samples of this form, to recover the secret vector s . This problem takes the form of a system of noisy linear equations modulo q :

$$\begin{aligned} b_1 &= a_{1,1}s_1 + a_{1,2}s_2 + \cdots + a_{1,n}s_n + e_1 \pmod{q}, \\ b_2 &= a_{2,1}s_1 + a_{2,2}s_2 + \cdots + a_{2,n}s_n + e_2 \pmod{q}, \\ &\vdots \\ b_m &= a_{m,1}s_1 + a_{m,2}s_2 + \cdots + a_{m,n}s_n + e_m \pmod{q}. \end{aligned}$$

Or equivalently,

$$b = As + e \pmod{q}.$$

If the error term e were absent, the system would consist of exact linear equations for each s_i in s , meaning that standard linear algebra techniques such as Gaussian elimination would efficiently recover the secret after collecting sufficiently many samples (n). The introduction of the error term makes the equations noisy, i.e., they are correct up to some small additive error. The numerical methods that would work with the error instead amplify the noise to unmanageable levels with every addition and especially multiplication, erasing the information vital to recover s and making it extremely computationally costly.

Solving the LWE problem has been proven to be as difficult as solving the worst-case hardness of problems in lattice theory, namely the decision version of the shortest vector problem (GapSVP) and the shortest independent vectors problem (SIVP). This lattice reduction underpins its use as a foundation for post-quantum cryptography, as it implies that if an algorithm can efficiently solve LWE, then there exist efficient quantum algorithms for approximate lattice problems.

2.2.2 RING LEARNING WITH ERRORS (RLWE)

While LWE provides a strong theoretical foundation, directly using LWE in cryptographic constructions leads to large key sizes and high computational cost. The Ring Learning With Errors (RLWE) problem adapts LWE to structured algebraic rings, enabling more compact and efficient constructions that still preserve hardness guarantees.

Let $R = \mathbb{Z}[x]/\Phi(x)$ be a polynomial ring modulo a cyclotomic polynomial $\Phi(x)$. In practice, many schemes choose $\Phi(x) = X^N + 1$ for N a power of 2, which yields a ring dimension of N . Further, define $R_q = R/qR \cong \mathbb{Z}_q[x]/\Phi(x)$. This constructs a finite set of polynomials that have degree less than $\deg(\Phi(x))$ and integer coefficients less than or equal to q .

An RLWE sample is generated analogously to LWE. A secret polynomial $s(x) \in R_q$ is chosen uniformly at random. A polynomial $a(x) \leftarrow R_q$ is also chosen randomly, as well as an error polynomial $e(x)$, whose coefficients are independently drawn from a small distribution. The RLWE sample out is:

$$(a(x), b(x)) = (a(x), a(x)s(x) + e(x) \bmod q).$$

The RLWE problem is to recover $s(x)$ given a sequence of independent samples of this form. The problem inherits worst-case hardness guarantees similar to LWE, based on reductions from hard lattice problems in ideal lattices. However, the polynomial ring structure in RLWE enables significantly more efficient arithmetic through polynomial multiplication, often implemented using the Number Theoretic Transform (NTT). This structure is what makes homomorphic encryption practically viable.

2.2.3 RLWE-BASED ENCRYPTION SCHEMES

In homomorphic encryption schemes, ciphertexts are constructed to resemble RLWE samples, with the secret key corresponding to the secret polynomial $s(x)$. The decryption

process effectively removes the linear term $a(x)s(x)$ from a sample to recover the underlying message, provided the accumulated noise introduced by the error remains sufficiently small, which is why it is important for HE schemes to effectively manage the noise growth under homomorphic operations.

The LWE and RLWE problems serve as hardness assumptions, and not cryptographic schemes themselves. In particular, practical homomorphic encryption schemes are built over the polynomial ring $\mathbb{Z}_q[x]/\Phi(x)$, where ciphertexts take the form of RLWE samples. This construction provides both the algebraic ring structure that enables homomorphic operations, as well as the security foundation that guarantees that uncovering a plaintext is computationally equivalent to solving an instance of RLWE.

2.3 THE CKKS SCHEME

The Cheon–Kim–Kim–Song (CKKS) encryption scheme, introduced in 2017 [9], enables approximate arithmetic over real-valued data, which is necessary in most machine learning applications. Unlike earlier homomorphic encryption schemes, such as BGV [6] and BFV [14], which support exact integer arithmetic and often require additional overhead to support real-number computations, CKKS is designed to efficiently handle approximate numerical computation.

This section presents a structured overview of the CKKS scheme, and it is based on the CKKS explanation series by OpenMined [27]. We begin with the encoding process, which is followed by the encryption and decryption procedures and their associated operations, and end with a discussion of practical CKKS implementation considerations. This discussion includes a detailed examination of the parameters that determine CKKS performance.

Figure 2.1 illustrates the overall information flow of the CKKS encryption scheme.

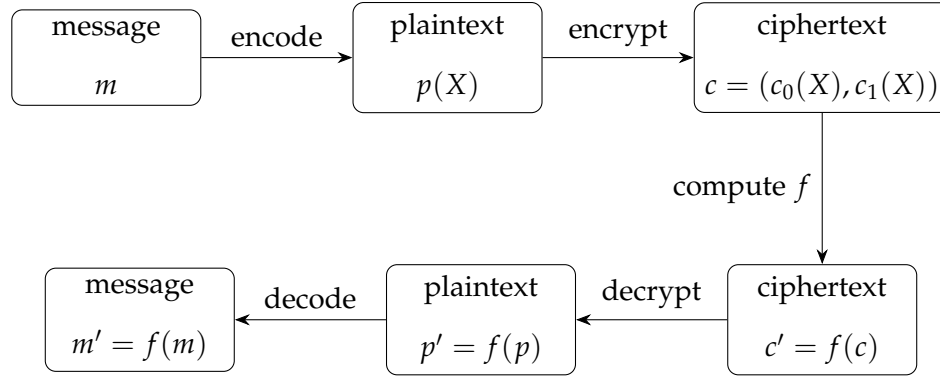


Figure 2.1: Schematic overview of the CKKS encryption scheme.

In Figure 2.1, m is the message we are trying to encrypt and f is a composition of homomorphic operations. CKKS supports three main types of operations that f can be comprised of: addition, multiplication, and rotation. Once we decrypt our resulting plaintext vector, we observe the homomorphic property that $\text{dec}(f(c)) = f(m)$. Table 2.1 describes the mathematical space that the data at each step exists in.

Step	Space	Description
Message	$m \in \mathbb{C}^{N/2}$	The vector of complex numbers to encode; packs numbers into up to $N/2$ slots.
Plaintext	$p(X) \in \mathbb{Z}[X]/(X^N + 1)$	A polynomial with integer coefficients reduced modulo $X^N + 1$; the ring where encoded messages live.
Ciphertext	$c \in (\mathbb{Z}_q[X]/(X^N + 1))^2$	A pair of polynomials with coefficients modulo q ; the ring supporting homomorphic operations where encrypted messages live.

Table 2.1: Description of data at each stage of the CKKS pipeline.

CKKS ensures that each mapping (encode/decode, encrypt/decrypt) has homomorphic properties, which is integral in ensuring that transforming data from step to step maintains error predictability and overall encryption accuracy. The main underlying mathematical structures behind the RLWE-based security and functionality of CKKS are

integer polynomial rings. Therefore, the first step in CKKS is to encode a complex vector m into an integer polynomial $p(X)$.

2.3.1 ENCODING

Data most often comes in the form of vectors of real numbers, rather than polynomials. The encoding-decoding step is vital to transform data into the polynomials that can be encrypted under CKKS. The goal of the encoding process is summarized as follows:

$$m \in \mathbb{C}^{N/2} \xrightarrow{\text{encode}} m(X) \in \mathbb{Z}[X]/(X^N + 1),$$

where N denotes the degree of our polynomial modulus $(X^N + 1)$, which is a power of 2 for CKKS. We will start by exploring a vanilla encoding technique that we will expand on to understand the actual encoding used in CKKS.

2.3.1.1 VANILLA ENCODING

The vanilla encoding encodes a vector $m \in \mathbb{C}^N$ into a polynomial $m(X) \in \mathbb{C}[X]/(X^N + 1)$.

Both the encoding and decoding procedure use the canonical embedding

$$\sigma : \mathbb{C}[X]/(X^N + 1) \rightarrow \mathbb{C}^{\mathbb{N}}.$$

This map sends a polynomial (mod $X^N + 1$) to its values at the complex roots of $\Phi_M(X)$, which is the M -th cyclotomic polynomial where $M = 2N$.

The n -th cyclotomic polynomial is the unique irreducible polynomial with integer coefficients that is a divisor of $x^n - 1$ and is not a divisor of $x^k - 1$ for any $k < n$. Since N is a power of 2 in CKKS, this corresponds to the case where

$$\Phi_M(X) = X^N + 1.$$

This polynomial is used in CKKS because of its nice algebraic structure. The roots of $(X^N + 1)$ are exactly the primitive M -th roots of unity with odd exponents. Thus, by letting $\xi = e^{2\pi i/M}$, we get that the roots of $X^N + 1$ are

$$\xi, \xi^3, \xi^5, \dots, \xi^{M-1}.$$

These points lie on the complex unit circle and are evenly spaced out, which gives us our desired algebraic structure for encoding, because rotations on the unit circle behave like multiplications in the polynomial ring modulo $\Phi_M(X)$

The idea behind decoding is now straightforward. To decode a polynomial $m(X)$ into a vector m , we evaluate $m(X)$ at the roots of $\Phi_M(X)$. This defines

$$\sigma(m(X)) = (m(\xi), m(\xi^3), \dots, m(\xi^{M-1})) \in \mathbb{C}^N.$$

The canonical embedding σ defines an isomorphism (a bijective homomorphism), so any polynomial has a unique vector, and vice versa. Going the other direction, encoding a vector into the corresponding polynomial, involves computing the inverse σ^{-1} . So, the problem is to, given a vector $m \in \mathbb{C}^N$, find a polynomial $m(X) = \sum_{i=0}^{N-1} \alpha_i X^i \in \mathbb{C}[X]/(X^N + 1)$ such that $\sigma(m(X)) = (m(\xi), m(\xi^3), \dots, m(\xi^{M-1})) = (m_1, m_2, \dots, m_N)$. To solve this, we set up the problem as the following system:

$$\sum_{j=0}^{N-1} \alpha_j (\xi^{2i-1})^j = m_i, \text{ for } i = 1, 2, \dots, N.$$

This is a linear equation of the following form:

$$A\alpha = m,$$

where A is the Vandermonde matrix of $(\xi^{2i-1})_{i,\dots,N}$, m is the vector we want to encode,

and α is the vector of the polynomial coefficients satisfying the constraints of the problem. We get that $\alpha = A^{-1}m$, which lets us define

$$\sigma^{-1} = \sum_{i=0}^{N-1} \alpha_i X^i \in \mathbb{C}[X]/(X^N + 1).$$

In this example, we see that the canonical embedding σ allows us to simply decode a polynomial by evaluating it on the roots of $X^N + 1$. This gives us an isomorphism between $\mathbb{C}^N$ and $\mathbb{C}[X]/(X^N + 1)$. However, we want the encoder to output polynomials in an integer polynomial ring, as that is the necessary structure for CKKS, so we can build on this vanilla encoder to provide the desired output.

2.3.1.2 CKKS ENCODING

The CKKS encoder needs to map complex input vectors as follows:

$$m \in \mathbb{C}^{N/2} \xrightarrow{\text{encode}} m(X) \in \mathbb{Z}[X]/(X^N + 1),$$

We now want the plaintext space to be $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$, which is the polynomial ring where each encoded polynomial element has integer coefficients.

Let us consider the image of the canonical embedding σ on $\mathcal{R}$. Recall that σ evaluates a polynomial at complex roots. Since polynomials in $\mathcal{R}$ have integer coefficients, we have a real-valued polynomial being evaluated at complex numbers. This means that evaluation at conjugate inputs produces conjugate outputs, i.e. $f(\bar{z}) = \overline{f(z)}$. Given the conjugate-pair symmetry of the complex unit circle, the root list of $X^N + 1$ contains each root and its conjugate. Thus, the image of $\sigma(\mathcal{R})$ is fully determined by only $N/2$ evaluations, and therefore does not produce an arbitrary vector in $\mathbb{C}^N$, but rather one with conjugate symmetry. This subring $\mathbb{H}$ is defined as follows:

$$\sigma(\mathcal{R}) \subseteq \mathbb{H} = \{z \in \mathbb{C}^N : z_j = \overline{z_{-j}}\}.$$

Any element in $\sigma(\mathcal{R})$ is in a space of dimension $N/2$, not N , which is why in CKKS encoding (σ^{-1}), the complex vectors of size $N/2$ have to be expanded by copying the other half of conjugate roots.

CKKS formalizes this operation with a projection map

$$\pi : \mathbb{H} \rightarrow \mathbb{C}^{N/2},$$

which extracts the $N/2$ independent coordinates. With a vector $z \in \mathbb{C}^{N/2}$, it can be expanded to fill in the conjugate half using π^{-1} to get $\pi^{-1}(z) \in \mathbb{H}$. Additionally, recall that σ^{-1} takes a conjugate-symmetric vector and produces the corresponding polynomial. The CKKS function pipeline is now summarized as follows:

$$\mathbb{C}^{N/2} \xrightarrow{\pi^{-1}} \mathbb{H} \xrightarrow{\sigma^{-1}} \mathbb{Z}[X]/(X^N + 1)$$

However, a key issue we need to consider is that the image of σ is strictly inside of the codomain of π^{-1} , i.e. $\sigma(\mathcal{R}) \subset \mathbb{H}$, meaning that an element of $\mathbb{H}$ is not necessarily an element of $\sigma(\mathcal{R})$. In other words, for an arbitrary vector $z \in \mathbb{C}^{N/2}$, $\pi^{-1}(z)$ is not necessarily the evaluation of any polynomial with integer coefficients. We therefore have to project $\pi^{-1}(z) \in \mathbb{H}$ onto $\sigma(\mathcal{R})$.

Since $\mathcal{R}$ has an orthogonal $\mathbb{Z}$ -basis $\{1, X, X^2, \dots, X^{N-1}\}$ and σ is an isomorphism, $\sigma(\mathcal{R})$ has an orthogonal $\mathbb{Z}$ -basis given by $\beta = (b_1, b_2, \dots, b_{N-1}) = (\sigma(\beta_1), \sigma(\beta_2), \dots, \sigma(\beta_{N-1}))$. For any $z \in \mathbb{H}$, we can then project on β with:

$$\text{proj}(z) = \sum_{i=1}^N x_i b_i, \text{ with } x_i = \frac{\langle z, b_i \rangle}{\|b_i\|^2} \in \mathbb{R}.$$

We are using the Hermitian product in the above projection to give us real outputs. The coordinates z_i need to be rounded to the higher or lower closest integer to give us a

polynomial with integer coordinates in the basis $(\sigma(\beta_1), \sigma(\beta_2), \dots, \sigma(\beta_{N-1}))$, which will belong to $\sigma(\mathcal{R})$.

The rounding procedure might destroy significant numbers, which is why in CKKS we need to multiply by a scaling factor $\Delta > 0$ during encoding and then divide by Δ during decoding. This ensures that we keep a precision of $\frac{1}{\Delta}$.

Consider the case of rounding $x = 1.4$. Instead of rounding to the nearest integer, we want to round to the closest multiple of 0.25 to maintain some precision. We can then set the scaling factor $\Delta = 4$, giving us a precision of 0.25. Now, $\lfloor \Delta \cdot x \rfloor = \lfloor 4 \cdot 1.4 \rfloor = \lfloor 5.6 \rfloor = 5$, which is an integer, as needed for CKKS. Then, once we divide by the same Δ during decoding, we get 1.25, which is the closest multiple of 0.25 to $x = 1.4$.

The entire encoding procedure can be summarized as follows:

$$\mathbb{C}^{N/2} \xrightarrow{\pi^{-1}} \mathbb{H} \xrightarrow{\Delta} \mathbb{H} \xrightarrow{proj} \sigma(\mathcal{R}) \xrightarrow{\sigma^{-1}} \mathcal{R}.$$

We can then decode a polynomial $m(X)$ by taking $m = \pi \circ \sigma(\frac{1}{\Delta} \cdot m(X))$.

2.3.2 ENCRYPTION AND DECRYPTION

CKKS supports approximate arithmetic, and not exact arithmetic, in the sense that the final computation on the encrypted data result might differ slightly from the result if the computations are performed directly on the unencrypted data. For example, adding 5 and 4 (both encrypted) might result in 8.99 or 9.01 when decrypted. Fully homomorphic schemes like BFV are exact, meaning they will always output 9. The benefit of using CKKS comes from working with arithmetic on real numbers in the context of computer representations, where results can be approximate but reasonably close.

CKKS is a public key encryption scheme, so a public and private key are generated. The public key is shared openly, while the private key is kept secret by the owner. The data encrypted using the public key can only be decrypted with the corresponding private

key, ensuring that only the intended recipient can read a message, even if the data is sent through an insecure channel.

The foundation behind CKKS encryption is the Ring Learning With Error (RLWE) problem. Recall that the CKKS encoder encodes a vector of complex values of size $\mathbb{C}^{N/2}$ to a corresponding polynomial in $\mathbb{Z}[X]/(X^N + 1)$, which is then scaled for precision. Therefore, the RLWE-based CKKS encryption works on a scaled polynomial, not a vector of numbers, i.e. $\Delta \cdot p(X)$.

A key component of a leveled homomorphic encryption scheme is to manage the size of messages, as computing a circuit of multiplicative depth L without rounding messages would result in the bit size of the output growing exponentially with L . After L multiplications, the encoded message m would become $m \cdot \Delta^L$, which would require an impractically large modulus q to hold the exponentially large coefficients, because if it were not large enough, the coefficients would be wrapped around by the modulus, destroying the underlying encoded message. CKKS overcomes this challenge by introducing a rescaling technique that divides intermediate values by a fixed base, allowing some of the Least Significant Bits (LSB) to be discarded while the error is still kept relatively small. This technique keeps the size of the messages ($m \cdot \Delta$) roughly the same and makes the required size of the modulus grow linearly with L .

To ensure security and precision, some parameters are fixed in CKKS: a base $p = 2$, an integer $N = 2^k$, which determines the size of the cyclotomic polynomial, and a modulus q_0 . Since RLWE computes in a finite ring, CKKS introduces a coefficient modulus defined as a chain of $L + 1$ moduli $q = q_0 q_1 \dots q_L$, satisfying that $q_\ell = p^\ell q_0$ for $0 < \ell \leq L$. Thus, at a multiplicative level $0 \leq \ell \leq L$, CKKS works in the polynomial ring

$$\mathcal{R}_q = \mathbb{Z}_{q_\ell}[X]/(X^N + 1).$$

The first step in CKKS is to generate the keys at the top level L . A small secret polynomial

s is sampled from the set of signed binary vectors in $\{0, \pm 1\}^N$ with a predetermined exact Hamming weight that is determined by the desired security level and the top modulus q_L . This polynomial will be used for the secret key only available to the owner.

A random polynomial a is uniformly drawn from $\mathcal{R}_{q_L}$, and a small error polynomial e is drawn by sampling coefficients from a discrete Gaussian noise distribution. Recall that solving RLWE is a problem of distinguishing noisy pairs of polynomials from truly random ones. Thus, we construct the private and public keys via the following formulation:

$$SK = (1, s) \in \mathcal{R}^2$$

$$PK = (b, a) = (-a \cdot s + e \pmod{q}, a) \in \mathcal{R}_q^2,$$

where $a \cdot s$ denotes the polynomial product of s and a . This is where we see one of the benefits of basing CKKS on RLWE because, since all operations are done between polynomials, both the private and public keys are of size $\mathcal{O}(n)$. Multiplications are done on polynomials, and can therefore be done with a complexity of $\mathcal{O}(n \log(n))$ using Discrete Fourier Transform.

The final key that is generated at the top level L is the evaluation key used in homomorphic multiplications. An additional integer P is chosen as a function of the desired security level and q_L . With this, we sample an additional polynomial a' from $\mathcal{R}_{P \cdot q_L}$ and an additional error polynomial e' following the same distribution as before. Then, the evaluation key becomes the pair

$$EK = (b', a') = (-a' \cdot s + e' + P \cdot s^2 \pmod{P \cdot q_L}, a') \in \mathcal{R}_{P \cdot q_L}^2.$$

To encrypt an encoded message m (a polynomial), we use $PK = (b, a)$ and $\mathcal{ZO}(\rho)$, which for a $0 \leq \rho \leq 1$ draws each entry in the vector from $\{0, \pm 1\}^N$ with probability $\rho/2$ for each ± 1 , and probability $1 - \rho$ for 0. We then sample an ephemeral vector $v \leftarrow \mathcal{ZO}(0.5)$ and error terms e_0, e_1 from a discrete Gaussian distribution. CKKS then

encrypts m with the following formulation:

$$c = (c_0, c_1) = (v \cdot PK + (m + e_0, e_1) \pmod{q_L}) \in \mathcal{R}_q^2.$$

Then, to decrypt a message $c = (c_0, c_1)$ using the secret polynomial s , we evaluate

$$c_0 + c_1 \cdot s \pmod{q_\ell},$$

which will be equal to the encoded message plus some predetermined noise.

Now that we have laid out the foundation of CKKS via the key generation, encryption, and decryption procedures, we can go into how addition, multiplication, and rescaling work. Adding two ciphertexts, $c_1 = (b_1, a_1)$ and $c_2 = (b_2, a_2)$, in the same modulus level ($c_1, c_2 \in \mathcal{R}_{q_\ell}^2$) is defined in the following component-wise and trivial manner:

$$c_{add} = c_1 + c_2 = (b_1 + b_2, a_1 + a_2) \pmod{q_\ell}$$

We can see that this yields our desired output when c_{add} is decrypted under CKKS,

$$(b_1 + b_2) + (a_1 + a_2) \cdot s = (b_1 + a_1 \cdot s) + (b_2 + a_2 \cdot s) = c_1 + c_2 + 2e \approx c_1 + c_2.$$

The noise growth for ciphertext additions is negligible because the modulus q_ℓ is significantly larger than the small doubled noise term that addition introduces.

With multiplication, let us first look at the case of multiplying a ciphertext with a plaintext, which is the simpler option. We start with two plaintexts (encoded polynomials) μ and μ' . We then encrypt μ to get a ciphertext $c = (c_0, c_1)$ and can define the ciphertext-plaintext multiplication as follows:

$$c_{mult} = \mu' \cdot c = (\mu' \cdot c_0, \mu' \cdot c_1).$$

We can see that this yields our desired output when c_{mult} is decrypted under CKKS,

$$(\mu' \cdot c_0) + (\mu' \cdot c_1) \cdot s = \mu' \cdot (c_0 + c_1 \cdot s) = \mu' \cdot (\mu + e) = \mu' \cdot \mu + \mu' \cdot e \approx \mu' \cdot \mu.$$

Let us now look at the case of ciphertext-ciphertext multiplication. We have two ciphertexts $c_1 = (b_1, a_1)$, $c_2 = (b_2, a_2) \in \mathcal{R}_{q_\ell}^2$. The multiplication operation must satisfy that

$$dec(c_1, s) \cdot dec(c_2, s) = (b_1 + a_1 \cdot s) \cdot (b_2 + a_2 \cdot s) = b_1 b_2 + (b_1 a_2 + b_2 a_1) \cdot s + a_1 a_2 \cdot s^2.$$

By letting $d_0 = b_1 b_2$, $d_1 = (b_1 a_2 + b_2 a_1)$, $d_2 = a_1 a_2$, we get

$$d_0 + d_1 s + d_2 s^2.$$

With this in mind, decrypting in CKKS can actually be thought of as a polynomial evaluation of the secret key s . In this way, the polynomial is $c_0 + c_1 S$, where S is the polynomial variable. Thus decrypting $c = (c_0, c_1)$ is plugging s in the above polynomial.

Now, we can view the decryption operation on the product of two ciphertexts as the evaluation of the second-degree polynomial $d_0 + d_1 s + d_2 s^2$ at the secret key s . We can then naively define the corresponding multiplication of c_1 and c_2 as $c_{mult} = (d_0, d_1, d_2)$.

However, in doing so, we increased the size of the ciphertext. Ciphertexts were a pair of polynomials, but with this multiplication, we get a triplet. Therefore, if we continue multiplying, we would need increasingly more polynomials in our ciphertext. To fix this exponential growth, CKKS implements a relinearization procedure.

2.3.2.1 RELINEARIZATION

The goal of relinearization is to compute the additional $d_2 \cdot s^2$ term using only a polynomial of degree 1 during decryption. To do this, we need to find a pair of

polynomials (d'_0, d'_1) such that

$$\text{dec}((d'_0, d'_1), s) = d'_0 + d'_1 \cdot s = d_0 + d_1 \cdot s + d_2 \cdot s^2 = \text{dec}(c_1, s) \cdot \text{dec}(c_2, s).$$

This step allows us to have a couple instead of a triple, and it eliminates the need for the decryption evaluation to require the square of the secret key s . This keeps the ciphertext size constant through multiplications and maintains the same decryption circuit.

To have the relinearization procedure accomplish this, CKKS utilizes the generated evaluation key after every ciphertext-ciphertext multiplication. We will define E as a polynomial couple such that

$$\text{dec}(E, s) = d_2 \cdot s^2.$$

We can then define $(d'_0, d'_1) = (d_0, d_1) + E$, which guarantees that we get the desired second-order polynomial when decrypted:

$$\text{dec}((d'_0, d'_1), s) = \text{dec}((d_0, d_1), s) + \text{dec}(E, s) = d_0 + d_1 \cdot s + d_2 \cdot s^2,$$

The way to produce R in CKKS starts with choosing a large integer P as a function of the desired security level and the top-level modulus q_L . The evaluation key is constructed by sampling $a' \leftarrow \mathcal{R}_{P \cdot q_L}$ and an e' polynomial from a discrete Gaussian distribution. With these, the evaluation key is defined as follows to ensure that we get the s^2 reduction during multiplication:

$$EK = (b', a') = (-a' \cdot s + e' + P \cdot s^2 \pmod{P \cdot q_L}, a') \in \mathcal{R}_{P \cdot q_L}^2.$$

Using EK , we define the polynomial couple as

$$R = \lfloor P^{-1} \cdot d_2 \cdot EK \rfloor \pmod{q_\ell}.$$

Dividing by P acts as reducing the noise introduced with multiplying by d_2 , and we round the resulting coefficients to the nearest integer. Notably, we then work with modulo q_ℓ and not $P \cdot q_L$, as done when generating EK . Then, we can define ciphertext-ciphertext multiplication between c_1 and c_2 at modulus level ℓ as

$$c_{mult} = (d_0, d_1) + R.$$

This is why it is called relinearization, as the non-linear term (s^2) is collapsed to ensure decryption consistency and maintain ciphertext size.

The final operation integral to the CKKS scheme is the rescaling procedure. Recall that when we are working with encoded messages, the underlying values are scaled by Δ , meaning that with a single multiplication, the scale doubles

$$(m_1 \cdot \Delta) \cdot (m_2 \cdot \Delta) = m_1 m_2 \cdot \Delta^2$$

Therefore, the goal of rescaling is to keep the scale constant and reduce the noise in the ciphertext.

2.3.2.2 RESCALING

Suppose we must do a total of L ciphertext-ciphertext multiplications, and we have a scale Δ (a power of 2). Then we can define our top modulus q_L , which determines the total capacity of the multiplications, as

$$q_L = \Delta^L \cdot q_0.$$

We have to be careful not to overflow the modulus chain, which is why q_0 is chosen such that $q_0 \geq \Delta$. This inequality will represent how many bits we want before the decimal. For example, if we want 40 bits for the decimal part and 10 bits for the integer part, then

we would set $\Delta = 2^{40}$, $q_0 = 2^{\text{\#bits integer}} \cdot 2^{\text{\#bits decimal}} = 2^{10+40} = 2^{50}$. We can follow this same construction to define the modulus q_ℓ for any $0 \leq \ell < L$.

With these fixed parameters, the rescaling procedure takes a ciphertext at modulus level ℓ and rescales it to fit inside a lower modulus level ℓ' . It does this by scaling $\frac{q_{\ell'}}{q_\ell}$ to the polynomial entries of c and rounding the coefficients to the closest integers. It is defined as

$$c' = \left\lfloor \frac{q_{\ell'}}{q_\ell} c \right\rfloor \pmod{q_{\ell'}} \in \mathcal{R}_{q_{\ell'}}^2.$$

In most cases, the level is reduced by 1, so $\ell' = \ell - 1$. Now, because we defined $q_\ell = \Delta^\ell \cdot q_0$, this simplifies to

$$c' = \left\lfloor \frac{1}{\Delta} \cdot c \right\rfloor \pmod{q_{\ell'}}.$$

By formatting the modulus chain this way, when we multiply two ciphertexts and rescale, we get a result scaled by Δ , not Δ^2 . We also divide both of the underlying values plus noise, which reduces noise growth.

A central issue that CKKS addresses is that operations are performed with huge moduli. In the previous bit-precision example, if we wanted to perform L multiplication, this would result in $q_L = 2^{440}$, so polynomials in the space would result in computations that do not fit within a standard 64-bit system. To address this issue, CKKS uses the Chinese Remainder Theorem (CRT). For pairwise coprime numbers $p_1, \dots, p_L$, the CRT gives the ring isomorphism

$$\mathbb{Z} / \left(\prod_i^L p_i \right) \mathbb{Z} \cong \prod_i^L \mathbb{Z} / p_i \mathbb{Z}.$$

The isomorphism is given by the map

$$x \pmod{p} \mapsto (x \pmod{p_1}, \dots, x \pmod{p_L}).$$

The idea behind using this theorem is that arithmetic on the larger ring can instead be performed independently on the smaller rings, which fit within 64-bit systems.

In CKKS, this translates to instead of having $q_L = \Delta^L \cdot q_0$, we first choose $p_1, \dots, p_L, q_0$ prime numbers such that each $p_\ell \approx \Delta$, and that q_0 is a prime number greater than Δ to reflect the bits of desired integer precision. Using these, we define the top-level modulus to be

$$q_L = \prod_{\ell=1}^L p_\ell \cdot q_0.$$

This way, we can use the CRT to perform operations with large moduli. We get that the rescaling procedure in CKKS is

$$c' = \left\lfloor \frac{q_{\ell-1}}{q_\ell} c \right\rfloor \pmod{q_{\ell-1}} = \left\lfloor \frac{1}{p_\ell} \cdot c \right\rfloor \pmod{q_{\ell-1}}.$$

The rescaling algorithm divides a plaintext by a prime integer to remove some inaccurate LSB. The magnitude of the messages can then be maintained roughly the same throughout homomorphic operations, and thus the required size of the largest modulus grows linearly with circuit depth while the error is predictably bounded.

To multiply two ciphertexts in CKKS, three separate algorithms need to run to ensure that decryption using the secret key will provide the correct result, which is illustrated in Figure 2.2.



Figure 2.2: Computational pipeline for homomorphic multiplication in CKKS.

2.3.3 CKKS IMPLEMENTATIONS

While CKKS is defined at the mathematical level, practical implementations require additional considerations. This subsection presents techniques for improving computa-

tional efficiency, introduces practical security notions, and surveys commonly used CKKS libraries.

2.3.3.1 PRACTICAL OPTIMIZATIONS

Although CKKS is defined algebraically over the ring $\mathbb{Z}_q[x]/(X^N + 1)$, practical implementations rely on two critical optimizations to make them viable for real-world computer systems: the Residue Number System (RNS) and the Number Theoretic Transform (NTT).

Under RNS, large integers are represented as a set of remainders (residues) modulo a set of pairwise coprime integers according to the Chinese Remainder Theorem (CRT). By splitting large coefficients into smaller ones (< 64 bits), it enables fast, parallelizable approximate operations. In the context of CKKS, all operations are done modulo Q to ensure that the coefficients remain in the ring. RNS-based implementations decompose Q into $q_0, q_1, \dots, q_L$, which are mutually co-prime. Operations can then be performed independently modulo q_i , which are significantly smaller in size.

Moreover, modern CKKS implementations choose each prime q_i to be an NTT-compatible prime with a small word-length. NTT is a variant of the Discrete Fourier Transform (DFT) that operates over finite fields or rings. As a result, an efficient NTT-based polynomial multiplication method with the time complexity of $\mathcal{O}(n \log n)$ is used, rather than the rudimentary $\mathcal{O}(N^2)$.

However, RNS's reliance on the CRT introduces additional pre-processing and post-processing operations, requiring the need to optimize the hardware building blocks for these steps. Otherwise, the complexity of the RNS system will negate the advantages of parallelism. Optimizing NTT-RNS-based architectures is a field of ongoing research effort, with recent efforts proposed in [33].

A defining feature of CKKS is its support for SIMD (Single Instruction, Multiple Data). It accomplishes this through packing, which is a method that encodes a vector

of approximate real numbers into a single plaintext polynomial, which can then be encrypted into a single ciphertext. The maximum size of such a vector is determined by the degree parameter N , which allows for a vector of $N/2$ slots. Homomorphic operations are applied componentwise across slots, enabling vectorized computations within a single ciphertext, including additions, multiplications, and rotations. This batching mechanism dramatically improves throughput and is essential for practical large-scale encrypted computation, such as encrypted inference in AI systems. A recent study leveraged the SIMD capabilities of CKKS to present new efficient solutions for ranking, order statistics, and sorting within the CKKS scheme [23].

2.3.3.2 SECURITY

Security in public-key cryptography is completely dependent on the underlying mathematical problem. In the case of the CKKS scheme, this is the assumed hardness of the RLWE problem. The security level of a CKKS scheme is therefore defined as the computational cost of the best currently known attack on the cryptographic primitive (RLWE) [25].

In CKKS, and lattice-based homomorphic schemes in general, n -bit security refers to the cost of the best current known attack T being:

$$T > 2^n.$$

The estimated computation cost of current attacks are derived using lattice reduction algorithms like BKZ (Block-Korkin-Zolotarev) [21], and they therefore reflect the current state of cryptanalytic knowledge rather than an absolute lower bound.

The most widely-used tool for modeling the cost of the best known attacks on LWE/RLWE instances is the LWE Estimator introduced by Albrecht et al. [1]. Given the CKKS parameter set (N, Q, σ) , where σ is the standard deviation of the discrete Gaussian

distribution used to generate noise for key generation and encryption, the LWE Estimator outputs the estimated attack cost 2^λ over various attack models (including BKZ). Thus, the CKKS instance has λ -bit security.

In practice, the parameter set (N, Q) is chosen according to security tables that provide pre-computed safe parameter ranges (σ is typically fixed across instances). These tables freeze dynamic analysis results from tools like the LWE Estimator to produce practical guidelines, as running a full lattice-based security estimator is computationally intensive during runtime. The Homomorphic Encryption Security Standard [2] contains standardized tables that provide conservative upper bounds on permissible modulus sizes Q for fixed polynomial degrees N .

2.3.3.3 CKKS LIBRARIES

A crucial component to making homomorphic encryption practical is the availability of high-quality software libraries that implement these schemes for use in real systems. Over the past decade, several open-source and research-driven libraries have emerged to support homomorphic encryption, including the CKKS scheme.

There are four primary, actively-maintained libraries that implement the CKKS scheme: Microsoft SEAL [8], OpenFHE[4], HELib [18], HEAAN, and Lattigo [26]. Except for Lattigo, which is written in Go, these are all C++ libraries, and they vary in structure, supported features, and performance trade-offs. Another popular library is TenSEAL [5], which is a Python wrapper for Microsoft SEAL.

Below is a list of explanations for features that a library might support.

- **Open-source:** Indicates whether the full source code of the library is publicly available under an open-source license, enabling inspection and modification.
- **Multithreading:** Indicates native support for parallel execution across multiple CPU

cores. Since homomorphic operations are computationally intensive, multithreading can significantly improve performance for large ciphertexts or batch workloads.

- **Bootstrapping:** Indicates the support of the bootstrapping technique that refreshes ciphertext noise to allow theoretically unbounded circuit depth. Without bootstrapping, computations are limited to a finite depth.
- **IND-CPA^D:** Indicates support for a variant of IND-CPA attack tailored to CKKS deployments, which was first introduced by Li and Micciancio in [20]. This reflects whether the implementation internally supports specialized security configuration beyond basic semantic security.
- **AVX512 Support:** Refers to built-in optimization using Advanced Vector Extensions (AVX-512), a SIMD instruction set available on certain modern CPUs. Such support enables faster NTT for polynomial arithmetic.
- **GPU Support:** Indicates whether the library provides acceleration using GPUs, which can substantially reduce runtime for large ciphertext operations.
- **Serialization:** Refers to built-in functionality for converting cryptographic objects (e.g., keys and ciphertexts) into storable formats, which is essential for distributed or production deployments.

A comparison of the main libraries and their feature support is shown in Table 2.2.

A comparative study of CKKS implementations in Microsoft SEAL, PALISADE (the predecessor of OpenFHE), HELib, and HEAAN was conducted in 2024 [32]. The researchers found that for shallow, leveled computations, SEAL achieved the lowest latency and smallest error. PALISADE and HEAAN perform similarly in many primitive operations. In terms of memory footprint, ciphertext sizes are comparable across libraries, though SEAL and PALISADE tend to use smaller public keys, and HELib larger. Overall,

Table 2.2: Comparison of CKKS implementations across popular FHE libraries.

Feature	Microsoft SEAL	OpenFHE	HElib	HEAAN	Lattigo
Open-source	Yes	Yes	Yes	No	Yes
Multithreading	No	Yes	Yes	Yes	Yes
Bootstrapping	No	No	Yes	Yes	Yes
IND-CPAD ^D	No	Yes	Yes	Yes	Yes
AVX512 Support	Yes	Yes	Yes	Yes	No
GPU Support	No	No	No	Yes	No
Serialization	Yes	Yes	Yes	Yes	Yes

the study suggests SEAL for leveled CKKS workloads and HEAAN/OpenFHE for high-depth or feature-rich applications.

2.3.4 CKKS PARAMETERS

CKKS implementations expose a small set of user-controlled parameters that govern the performance of a CKKS instance. There are three primary parameters that jointly determine the security, the multiplicative depth supported by the scheme, and the achievable numerical precision:

1. the polynomial modulus degree N ,
2. the coefficient modulus chain Q , and
3. the scaling factor Δ .

Importantly, the coefficient modulus Q is defined as a product of primes to enable efficient computations in RNS form:

$$Q = \prod_{i=0}^L q_i,$$

where each q_i is a prime number internally chosen according to the Chinese Remainder Theorem. The user specifies a list of bit-sizes $[s_0, \dots, s_L]$, which then corresponds to primes such that the size of $q_i \approx s_i$ bits. The composite modulus Q is represented internally in RNS form, allowing successive primes to be removed through rescaling after each homomorphic multiplication, which is why L theoretically corresponds to the multiplicative depth of the circuit.

Selecting the right parameters for CKKS is crucial for both efficacy and efficiency in real-world deployment scenarios. The configuration values are not independently selected, but rather involve solving a constrained design problem in which security, numerical precision, and computational cost are tightly coupled. Each parameter influences the feasible range of the others, and satisfying one objective (e.g., higher security or higher precision) often comes at the expense of another (e.g., reduced security or increased runtime).

Crucially underlying the parameter selection process is the circuit being evaluated, as a bigger multiplicative depth requires enlarging the total modulus Q , which in turn reduces the security level for a fixed polynomial degree N . Restoring the desired security level then requires increasing N , which subsequently increases computational cost and memory usage. Similarly, increasing the scaling factor Δ to improve precision enlarges intermediate values, necessitating larger primes in the modulus chain, which propagates increased noise growth. Thus, CKKS parameter selection fundamentally involves balancing competing objectives.

The interdependency of CKKS parameters and the application-specific requirements of target security levels, depth of the circuit, and the output precision requirement is represented in Figure 2.3, and Table 2.3 interprets the effects.

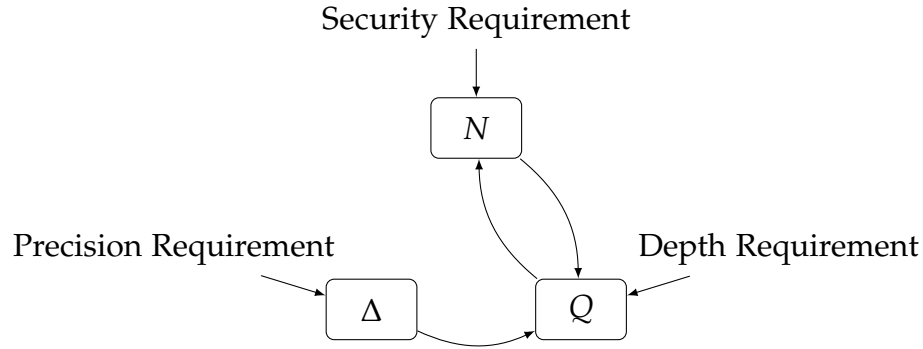


Figure 2.3: Dependency structure between CKKS parameters and application requirements.

Table 2.3: Dependency relationships between CKKS parameters and application requirements.

Dependency	Interpretation	Effect
Security $\rightarrow N$	Target bit-security determines the minimum ring dimension	$\uparrow \text{security} \implies \uparrow N$
Depth $\rightarrow Q$	Multiplicative depth determines the required modulus chain length	$\uparrow \text{depth} \implies \uparrow \text{len}(Q)$
Precision $\rightarrow \Delta$	Desired output precision determines the global scaling factor	$\uparrow \text{precision} \implies \uparrow \Delta$
$N \rightleftharpoons Q$	Larger modulus sizes can require larger ring dimension to preserve a security level	$\uparrow Q \implies \uparrow N$
$\Delta \rightarrow Q$	Larger scale requires larger moduli during rescaling to maintain precision	$\uparrow \Delta \implies \uparrow \text{sum}(Q)$

CHAPTER 3

RELATED WORK

The parameters chosen for a specific instance of the CKKS scheme significantly impact the performance of encrypted machine learning inference, in particular: security, precision, and computational cost. Choosing optimal parameters is a non-trivial task, and there is an ongoing research effort to abstract the process to reduce the expertise required for the practical and effective application of homomorphic encryption. This chapter provides an overview of the current state of CKKS parameter selection, highlighting the provided parameter functionality of CKKS libraries, as well as the current automation and simplification research efforts and tools. Finally, this chapter presents the motivation for the tool proposed in this work.

3.1 PARAMETER SELECTION IN CKKS LIBRARIES

Modern fully homomorphic encryption libraries that implement the CKKS scheme expose parameter configuration primarily at a low level. Libraries such as Microsoft SEAL, OpenFHE, and Lattigo provide the core primitives required to perform encrypted computation (e.g. multiplication and relinearization) while ensuring that user-provided parameter sets satisfy the 128-bit security minimum defined in the Homomorphic Encryption Standard [2].

However, beyond implementing security safeguards, CKKS libraries do not automate

parameter selection. Developers must manually choose the key parameters: polynomial modulus degree, coefficient modulus chain, and scaling factor. This process requires expertise and an understanding of how these parameters interact with the underlying circuit being evaluated under encryption.

Consequently, these libraries frequently serve as backend executors for higher-level frameworks that aim to automate encrypted computation pipelines [35]. While this modular design provides flexibility, it leaves the complexity of parameter selection largely to the developer. In the context of privacy-preserving ML, this introduces an extremely steep learning curve to efficiently implement homomorphic encryption.

3.2 COMPILER-BASED SYSTEMS FOR ENCRYPTED MACHINE LEARNING

To address the intricacy of applying homomorphic encryption in practical settings, several research systems have worked to raise the abstraction level. Systems such as CHET [10], Orion [12], and EVA [11] translate neural network models into homomorphic circuits and automatically derive parameters sufficient for encrypted inference.

While these compiler-based approaches remove much of the manual reasoning required to configure homomorphic encryption schemes, they are limited to a narrow scope of supported model architectures and use a one-shot framework to generate a single parameter configuration, which reduces the flexible exploration of parameter trade-offs across different security, precision, and performance requirements [35]. The compiler design is effective for fixed workloads with a white-box AI model, but less equipped for users who need to reason about parameter choices across a broader set of deployment scenarios using outsourced proprietary cloud computation.

3.3 APPLICATION-AWARE SECURITY AND PARAMETERIZATION

Recent research has highlighted security challenges arising from parameter configurations based on incorrect modeling of homomorphic encryption applications. Alexandru et al. [3] identified a gap between the security guarantees of CKKS under CPA^D and how CKKS is instantiated in practice. They brought attention to the application-agnostic definition of IND-CPA^D causing user confusion when instantiating application-specific CKKS, which had led to attacks proposed in [17].

The authors introduce the concept of application-aware homomorphic encryption (AAHE), which formalizes how encrypted applications should be described and analyzed when selecting parameters. They also propose an application specification language (ASL) that provides structured guidelines for implementing the AAHE model to achieve IND-CPA^D security in popular libraries.

This work examines the parameter selection workflow recommendations of current CKKS libraries, such as OpenFHE and HELib. The libraries recommend a two-phase process. The first, offline phase, is when developers execute the full computation on representative test inputs to estimate noise growth and determine suitable scheme parameters. In the second phase, the same parameters are used to evaluate the actual encrypted data.

While this approach can produce tight noise estimations for a given parameter set, it assumes that the computational circuit performed during estimation is identical to the one executed during evaluation. Because this assumption is not enforced by libraries, inconsistencies between these phases can lead to security vulnerabilities and invalidate security assumptions.

Importantly, the authors point to the security concerns of ambiguous descriptions of circuit structure, input magnitude, and operational details, which all significantly affect noise growth in CKKS. Although the proposed framework provides a formal mechanism for describing applications and identifying potential vulnerabilities, it focuses on formal

security modeling rather than providing methods for generating parameter configurations that accurately reflect the application setting.

3.4 AUTOMATED PARAMETER SELECTION RESEARCH

Several research efforts have worked to automate parameter selection directly. The most prevalent approach involves treating parameter selection as a multi-objective optimization problem of precision, performance, and security. Cabrero-Holgueras and Pastrana [7] introduced an expert framework that combines fuzzy logic and linear programming to automatically generate parameter configurations.

Their system allows users to express preferences across three axes: security, precision, and performance, using a numerical scale. A fuzzy logic module then translates these preferences into coefficients used in a linear programming model to select parameters according to the Homomorphic Encryption Standard [2] and the user preferences.

The fuzzy logic step uses two modules: the Scale module, which estimates bounds for the polynomial modulus degree, and the Looseness module, which expands the feasible ranges for the optimization problem. These two modules output the coefficients used to define the constraints and objective function used in the linear programming step.

While this approach successfully reduces the manual parameter tuning process, its efficacy relies on heuristic estimations based on user preferences. Beyond multiplicative depth, the framework does not capture the application context, so the selected parameters can be misguided by user input.

3.5 PRACTICAL GUIDELINES FOR CKKS PARAMETER SELECTION

A recent effort proposed structured guidelines for CKKS parameter selection. Gharibyar et al. [16] experimentally analyzed the effect of parameter choices on performance and precision in common statistical computations performed under CKKS encryption.

The results of the study indicated that computational cost is largely determined by the ring dimension and multiplicative depth of the statistical function being evaluated, while numerical precision is primarily influenced by the scaling factor. Based on the findings, the paper provides practical recommendations for selecting parameters that fall within a desired security level.

This work serves primarily as a set of practical guidelines, and still requires substantial manual intervention from users when applying the CKKS scheme in real-world applications.

3.6 SUMMARY AND MOTIVATION

Existing work on CKKS parameter selection primarily focuses on abstracting the low-level cryptographic primitives offered by libraries and automating parameter generation through compilers designed for specific workloads. While these approaches reduce some of the complexity associated with homomorphic encryption, they often generate a single parameter configuration or require substantial user expertise to discern trade-offs between security, numerical precision, and computational performance.

As a result, there remains a gap in tools that allow developers to systematically explore parameter configurations that reflect application-specific requirements, particularly in practical cloud-based machine learning scenarios.

The tool presented in this work addresses this gap by providing an automated parameter selection framework built on top of existing CKKS libraries. The system incorporates security constraints defined by the Homomorphic Encryption Standard [2]

and integrates application-level requirements, including properties of the input data and the multiplicative depth of the underlying circuit. Using this information, the tool automatically generates and validates candidate parameter configurations that reflect the structure of the application.

Rather than producing a single configuration, the framework provides a set of parameter suggestions that capture different trade-offs between security, numerical precision, and computational cost. These alternatives enable users to better understand the design space of CKKS parameterization while requiring minimal prior knowledge or predetermined objective preferences.

CHAPTER 4

SYSTEM DESIGN

This chapter presents a parameter selection tool for the CKKS homomorphic encryption scheme that automates the parameter search and selection process. In contrast to existing parameter recommendation frameworks, the tool incorporates model-informed and data-informed constraints that reflect the structure of real-world workloads.

To motivate the tool’s utility, consider a realistic deployment scenario. A client, such as a business, hospital, or individual, possesses a sensitive dataset

$$X_{\text{client}} = \{x_1, x_2, \dots, x_n\}.$$

For example, if this were a U.S. hospital dataset, each x_i might represent a patient record containing sensitive clinical information such as diagnostic measurements, medical history, and treatment outcomes. Because these records are protected under the Health Insurance Portability and Accountability Act (HIPAA), the hospital must preserve patient confidentiality when outsourcing any data processing.

Now, suppose that this client wants to obtain predictions using a powerful, high-accuracy, and proprietary machine-learning model f hosted on an untrusted cloud platform. In the context of the hospital, this might be a model trained to identify patients likely to develop a specific medical condition. Directly sending plaintext data to the model introduces a myriad of security concerns: data interception during transit,

unauthorized inspection by the model provider, and leakage through server-side black-box computations.

A public-key homomorphic encryption is a cryptographic solution to this issue, as it would enable the secure interaction between the private data and the public model. Under the CKKS scheme, the client encrypts each input vector using a public key

$$c_i = \text{Enc}_{\text{pk}}(x_i)$$

and sends the resulting ciphertext c_i to the server. The server then evaluates the model homomorphically without having any access to the underlying plaintext

$$c'_i = \text{Eval}(f, c_i).$$

The client then decrypts the result c_i using the corresponding private key

$$y_i = \text{Dec}_{\text{sk}}(c_i),$$

which retrieves the model's prediction while preserving the confidentiality of the input data throughout the entire computation pipeline.

Although the information flow is straightforward, the security, accuracy, and efficiency of privacy-preserving machine learning depend on the chosen CKKS parameters: polynomial modulus degree N , ciphertext modulus chain Q , and scale Δ . Existing CKKS implementations provide generic default configurations, often requiring manual fine-tuning to find acceptable performance. The framework developed in this thesis addresses this gap by providing a comprehensive and automated approach to parameter selection. The tool performs a multi-objective search to recommend CKKS parameters optimized for the user-provided dataset and model. A central component of the tool is automatic precision-bit estimation using the underlying dataset, which enables the

CKKS parameter selection process to be fully data-driven rather than dependent on user-specified estimates of the required numerical precision.

The proposed tool follows a six-stage pipeline, which is presented in Figure 4.1 and described in Table 4.1.

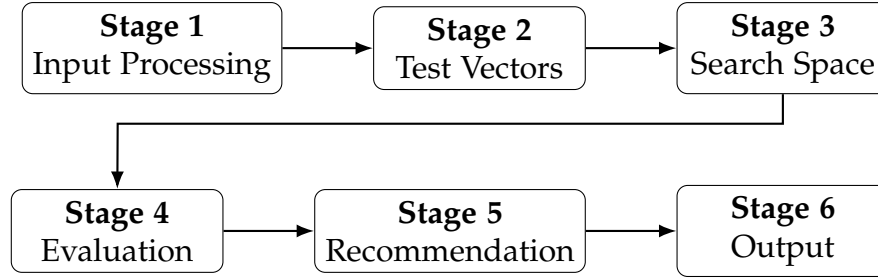


Figure 4.1: Six-stage pipeline for CKKS parameter selection.

Table 4.1: Description of the six-stage CKKS parameter selection pipeline.

Stage	Description
Input Processing	The pipeline begins with a user-provided dataset and trained model, which are used to determine the statistical and structural requirements for encrypted inference.
Test Vectors	Statistically representative test vectors are generated to approximate real inference constraints when evaluating candidate parameter sets.
Search Space	A structured search space of CKKS parameter configurations is constructed based on model depth, dataset precision requirements, and security constraints.
Evaluation	Each candidate parameter set is tested via encrypted inference, measuring precision loss and runtime.
Recommendation	A multi-objective optimization process that identifies the optimal CKKS configurations within the search space.
Output	The recommended parameters are validated through full encrypted inference, and a comprehensive experiment report is produced for the user.

4.1 INPUT PROCESSING

The tool requires two fundamental inputs: a real-valued dataset consisting of n samples and d features

$$\mathcal{D} \in \mathbb{R}^{n \times d},$$

and a pretrained model

$$f : \mathbb{R}^d \rightarrow \mathbb{R}.$$

The dataset must be numerically compatible with the model, in the sense that the model has been trained on data with the same feature structure and numerical ranges. Under this assumption, evaluating a plaintext entry $x \in \mathbb{R}^d$ through f produces a meaningful result. This in turn ensures that we can establish a baseline against which inference can be compared.

4.1.1 DATA-DRIVEN PRECISION ESTIMATION

From the dataset $\mathcal{D}$, the tool automatically analyzes its numerical structure to infer the precision requirements needed for CKKS encoding. Existing CKKS workflows typically require users to manually incorporate any desired bits of precision into the parameter choice for the scale Δ . However, these choices are often *ad hoc* and can lead to unnecessarily large scales or oversized modulus chains, reducing overall effectiveness and efficiency. By deriving the estimated integer and decimal bits of precision directly from the shape and distribution of the data, the tool establishes that the selected CKKS parameters are data-efficient.

The two precision quantities that need to be estimated are integer precision, which is how many bits are needed to represent the magnitude of the data, and decimal precision, which is how many bits are needed to represent the smallest meaningful difference between distinct values.

4.1.1.1 INTEGER PRECISION ESTIMATION

Given the dataset $\mathcal{D}$, let

$$M = \max_{x \in \mathcal{D}} \{|x|\}$$

be the largest absolute value appearing in the dataset. To encode all values without overflow, CKKS requires enough integer bits to represent M . This minimum number is calculated as

$$\text{bits}_{\text{int}} = \lceil \log_2(M) \rceil.$$

This lower bound ensures that the encoded plaintext polynomial can store every feature value in the dataset without truncation.

4.1.1.2 DECIMAL PRECISION ESTIMATION

CKKS represents real numbers by scaling them by a factor Δ and then rounding them to the nearest whole number. Therefore, if the difference between two real numbers is less than $\frac{1}{\Delta}$, they may round to the same encoded value. Let

$$\delta_{\min} = \min_{x_i, x_j \in \mathcal{D}} \{|x_i - x_j| : x_i \neq x_j\}$$

be the smallest nonzero gap between distinct feature values in the dataset. To ensure that CKKS can distinguish values separated by $\delta_{\min}$, the scale must satisfy

$$\Delta \geq \frac{1}{\delta_{\min}}.$$

From this inequality, because it is recommended to set $\Delta = 2^{\text{bits}_{\text{dec}}}$, the minimum required number of decimal bits is

$$\text{bits}_{\text{dec}} = \lceil -\log_2(\delta_{\min}) \rceil.$$

This guarantees that the encoded values preserve the resolution in the dataset.

These automatically estimated precision bits of bits_{int} and bits_{dec} are then used throughout the parameter-selection pipeline as lower bounds to ensure that the created CKKS configurations faithfully reflect the input data structure.

4.1.2 MODEL COMPATIBILITY CONSTRAINTS

A central requirement is that the model must be homomorphically compatible, meaning that its computation can be expressed using only additions and multiplications. Therefore, any non-linear functions that might be part of a standard machine learning model, such as ReLU or Sigmoid, must be replaced by polynomial approximations. This is often accomplished using low-degree Chebyshev polynomials. It is crucial that these homomorphically compatible approximations are incorporated into the training of the model, not just inference. For example, a logistic regression model trained using the true sigmoid function

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

will not remain consistent if its encrypted forward pass uses a corresponding polynomial approximation $\tilde{\sigma}(t)$. The same learned weights with mismatched activation functions lead to systemic errors after decryption, as it will not coincide with the plaintext forward pass. It is therefore essential that the encrypted and plaintext computation graphs perform equivalent operations.

Moreover, the encrypted forward pass must be implemented using the operations provided by the CKKS library. Standard numerical libraries like numpy cannot directly operate on encrypted ciphertext objects. Consequently, operations such as vector

dot products or matrix multiplications must be expressed using the corresponding homomorphic operations available through the encryption library.

Since CKKS noise grows with ciphertext-ciphertext multiplications, the multiplicative depth of the model plays a critical role in the parameter selection process. The tool thus requires a known multiplicative depth, which is maintained within the tool as a dedicated variable.

4.1.3 SIMULATION OF CLIENT-SERVER DEPLOYMENT

To support experimentation and to simulate realistic deployment scenarios, the tool provides built-in utilities to create a synthetic client-server data split. A pre-existing labeled dataset is partitioned into

$$X_{\text{client}}, X_{\text{server}}, y_{\text{server}},$$

where the server-designated portion is used to train a simple, homomorphic-compatible linear or logistic model. The labels are intentionally withheld from the client side to mirror the real-world context in which a client seeks predictions from a model it did not train. Because both splits share the same feature structure, the trained model remains valid for encrypted inference of client data, which was importantly not used in training.

4.1.4 BUILT-IN DATASETS AND MODEL CONSTRUCTION

The tool provides access to two standard datasets from the Python library `sklearn`: Breast Cancer Wisconsin dataset and the Diabetes dataset. The tool then allows for the partitioning process outlined earlier, and the resulting server data (X_{server} and y_{server}) can be used to train a linear regression model or a logistic linear regression model, depending on if the task is a linear regression or binary classification. These built-in

training utilities return models that satisfy the homomorphic-compatibility requirements and can be directly integrated into the parameter-selection pipeline.

4.2 TEST VECTOR CONSTRUCTION

The test vectors generated during this stage serve as inputs to evaluating candidate CKKS parameter sets. Rather than selecting raw samples from the client dataset, the tool constructs synthetic vectors that statistically resemble real client inputs. The purpose of using these synthetic test data is two-fold: (1) It preserves privacy by avoiding using actual sensitive client data, and (2) it ensures consistency, as the synthetic vectors capture the statistical characteristics that determine ciphertext magnitude, noise growth, and precision loss during encrypted inference.

Let the dataset consist of n samples and d features,

$$x_i = (x_{i1}, x_{i2}, \dots, x_{id}) \in \mathbb{R}^d, \quad i = 1, \dots, n.$$

For each feature $j \in \{1, \dots, d\}$, the tool computes four empirical statistics: mean (μ_j), standard deviation (σ_j), minimum (a_j), and maximum (b_j). These four quantities form the feature descriptor:

$$D_j = (\mu_j, \sigma_j, a_j, b_j).$$

To construct a representative test vector, the tool employs a default sampling method in which each feature is sampled independently from a truncated normal distribution:

$$\tilde{x}_j \sim \text{TruncNorm}(\mu_j, \sigma_j^2; a_j, b_j),$$

where the support is restricted to the empirical range $[a_j, b_j]$. This method provides a simple mechanism for generating inputs based on the empirical statistics of the dataset

and remain bounded within the feature ranges. The resulting synthetic test vector is

$$\tilde{\mathbf{x}} = (\tilde{x}_1, \tilde{x}_2, \dots, \tilde{x}_d).$$

This sampling process yields test vectors that reflect the magnitude and variability of the true client data while avoiding the use of any specific sensitive entry. Since CKKS precision loss depends heavily on the scale and structure of the input features, these synthetic vectors provide a reliable proxy for evaluating candidate parameter sets, which ensures that the recommended parameters support both correctness and efficiency under realistic operating conditions.

4.3 SEARCH SPACE GENERATION

This stage of the tool pipeline constructs a structured search space of candidate CKKS parameter sets. Each candidate is a triple

$$(N, \Delta, Q),$$

where N is the polynomial modulus degree, Δ the scaling factor, and

$$Q = q_0 q_1 \dots q_L$$

is the ciphertext modulus chain. Given the non-trivial interdependence between the parameters, the tool generates the search space by determining which parameter sets are admissible. The method takes three inputs, which come from the underlying dataset and model, as outlined in Section 4.1: the multiplicative depth of the model L_{model} , the desired integer precision (in bits), and the desired decimal precision (in bits).

From these inputs, the policy constructs candidate sets for: polynomial degrees $\mathcal{D}$,

scaling factors $\mathcal{S}$, and modulus chains $\mathcal{Q}$. The search space is the Cartesian product

$$\mathcal{D} \times \mathcal{S} \times \mathcal{Q}.$$

4.3.0.1 SELECTING CANDIDATE POLYNOMIAL DEGREES

Implementations of the CKKS scheme primarily support three standard polynomial degrees

$$N \in \{8192, 16384, 32768\},$$

each corresponding to different maximum security levels and maximum allowable modulus size.

The maximum multiplicative depth supported by a CKKS parameter set is determined by the total bit-capacity of the modulus chain. Each ciphertext-ciphertext multiplication consumes one RNS prime during rescaling, so the number of available primes bounds the depth. Because most CKKS implementations require each prime to be at least 30 bits, the maximum depth is approximately the total allowable modulus size divided by 30. Using the Homomorphic Encryption Standard guidelines that guarantee a 128-bit security minimum, which provide the maximum modulus size $|Q|_{\max}$ for each polynomial degree N , the maximum multiplicative depth can be computed as

$$L_{\max} = \left\lfloor \frac{|Q|_{\max}}{30} \right\rfloor.$$

The analytically determined maximum depth each degree can support is shown in Table 4.2.

Table 4.2: Maximum multiplicative depth for each polynomial modulus degree.

polynomial modulus degree	maximum depth
8192	7
16384	14
32768	29

Any degree N that has an associated maximum depth less than the model's multiplicative depth is omitted from the set of candidate polynomial degrees $\mathcal{D}$, ensuring that the search space contains only parameter sets capable of supporting the encrypted computation graph.

4.3.1 GENERATING CANDIDATE SCALES

CKKS implementations typically uses scales of the form

$$\Delta = 2^n, \quad n \in \{20, 30, 40, 50, 60\}.$$

The scale must be large enough to preserve the desired decimal precision throughout the computation, but not so large that it prematurely exhausts the modulus chain. To account for unpredictable noise growth, the default construction method introduces a buffer $\text{bits}_{\text{buffer}}$. The default value is

$$\text{bits}_{\text{buffer}} = 5.$$

Then, using the required decimal bits of precision estimated from the input dataset, the tool computes

$$\text{bits}_{\text{base}} = \text{bits}_{\text{dec}} + \text{bits}_{\text{buffer}},$$

where $\text{bits}_{\text{base}}$ is rounded to the nearest 10, and constructs candidate scales

$$\Delta_i = 2^{\text{bits}_{\text{base}} + i \cdot \text{scale}} \quad i \in \{0, 1, \dots, k\},$$

with the implementation having default values $\text{scale} = 10$ and $k = 2$. This construction ensures that the set of candidate scales $\mathcal{S}$ contains a controlled range of scales that balance precision and noise-budget consumption.

4.3.2 CONSTRUCTING CANDIDATE MODULUS CHAINS

For each (N, Δ) in $\mathcal{D} \times \mathcal{S}$, the tool constructs a set of candidate modulus chains. Recall that the modulus chain Q is divided into smaller prime integer moduli q_i , and that CKKS implementations allow for the specification of the bit sizes for each of the moduli.

4.3.2.1 CHOOSING THE FIRST MODULUS q_0

The first modulus q_0 must be large enough to encode the initial ciphertext without overflow. The tool computes a baseline using the dataset-informed estimates for the decimal and integer bits of precision:

$$q_{0_{\text{base}}} = \text{bits}_{\text{int}} + \text{bits}_{\text{dec}},$$

where $q_{0_{\text{base}}}$ is rounded to the nearest 10, and generates a set of candidates

$$Q_0 = \{q_0 : q_0 = \max(q_{0_{\text{base}}} + \delta, \Delta + \delta), \delta \in \{0, 10, 20\}\}.$$

This construction ensures that $q_0 > \Delta$, which is required for the initial ciphertext to not overflow. The increasing options from the minimum precision requirement accommodate different precision-noise tradeoffs, providing sufficient coverage.

4.3.2.2 CHOOSING THE CHAIN LENGTH L

Although the theoretical minimum chain length is the multiplicative depth of the computation graph L_{model} , practical CKKS implementations often require additional flexibility to account for unpredictable rescaling and rounding noise. Additionally, even depth-1 models (e.g., linear regression) require at least three moduli to support the entire encryption, multiplication, and decryption process.

The tool therefore tests default additional length buffers and ensures a minimum length:

$$\mathbb{L} = \{\max(3, L_{\text{model}} + \delta) : \delta \in \{2, 3\}\}.$$

4.3.2.3 CHOOSING THE FINAL MODULUS q_L

The final modulus q_L is often chosen to be larger than the internal moduli to preserve precision at the end of the computation. From theory, each modulus q_i should be set to approximately the same size as our global scale Δ . Thus, to ensure sufficient parameter space coverage, the tool generates a set of candidate final moduli while adhering to the standard maximum modulus size of 60 bits. For each $q_0 \in Q_0$ and $L \in \mathbb{L}$, the tool defines

$$q_{0_L} = \{q_0 + 10i \leq 60 : i \in \mathbb{N}_0\}.$$

4.3.2.4 CONSTRUCTING THE FULL MODULUS CHAIN

Each candidate chain has the form

$$(q_0, \underbrace{\Delta, \dots, \Delta}_{L-1}, q_L).$$

Using the generated candidate components, the full set of candidate chains is

$$\mathcal{Q} = \{(q_0, \Delta, \dots, \Delta, q_L) : q_0 \in Q_0, L \in \mathbb{L}, q_L \in q_{0_L}\}.$$

4.3.3 FINAL SEARCH SPACE

The complete generated search space is the Cartesian product

$$\mathcal{D} \times \mathcal{S} \times \mathcal{Q}.$$

Each element is a fully specified CKKS parameter set that accounts for: the model’s multiplicative depth, dataset precision requirements, CKKS structural constraints, and practical noise-growth buffers.

4.4 PARAMETER EVALUATION

Once the search space is generated, each candidate parameter set must be evaluated. The evaluation process takes four inputs: an encryption backend, the model, the synthetic test vectors, and the parameter set (N, Δ, Q) under consideration. For every candidate, the tool computes 4 metrics, as listed in Table 4.3, which together characterize the correctness, efficiency, and security of the parameter set.

Table 4.3: Evaluation metrics used in the parameter recommendation framework.

Metric	Category	Description
Precision	Empirical	Measures the error between encrypted and plaintext inference.
Runtime	Empirical	Captures the execution time of encrypted inference.
Security	Analytical	Quantifies the cryptographic security level of parameter choices.
Memory	Analytical	Estimates the memory footprint of the public-key cryptographic scheme.

4.4.1 EMPIRICAL EVALUATION METRICS

The empirical metrics are obtained by running the model’s encrypted inference on the synthetic test vectors. The tool uses a default encryption backend that supports TenSEAL, which is initialized using the candidate parameter set to create an encryption context. Each test vector $\vec{x}$ is encrypted, evaluated through the encrypted forward pass of the model, and then decrypted.

4.4.1.1 PRECISION

Precision measures how well encrypted inference approximates plaintext inference. For each test vector $\vec{x}$, the tool computes

$$e(\vec{x}) = \|\text{Dec}(f(\text{Enc}(\vec{x}))) - f(\vec{x})\|_1,$$

where the error is taken as the mean absolute difference across output coordinates. The precision score for the parameter set is the worst-case error across all test vectors:

$$\text{err}(N, \Delta, Q) = \max_{\vec{x}} \{e(\vec{x})\}$$

This metric captures the combined precision-loss effects of scaling, rescaling, and noise growth throughout the computation graph of f .

4.4.1.2 RUNTIME

Runtime measures the computational cost of encrypted inference. Using Python's `time` library, the tool records the average time required to encrypt the test vectors, evaluate the model homomorphically, and decrypt the outputs. For a test vector $\vec{x}_i$, the tool calculates

$$t(\vec{x}_i) = t_{\text{end}} - t_{\text{start}},$$

and then the final metric is computed as the average time over all n test vectors:

$$\text{time}(N, \Delta, Q) = \frac{1}{n} \sum_{i=1}^n t(x_i).$$

This metric reflects the impact of the sizes of N and Q , as well as the number of ciphertext operations performed during evaluation.

4.4.2 ANALYTICAL EVALUATION METRICS

The analytical metrics are properties of a given parameter set and do not require encrypted inference, as they are independent of any runtime processes. They capture the characteristics of a CKKS parameter set that affect security and memory usage.

4.4.2.1 SECURITY

The security in CKKS is governed by the hardness of the Ring Learning With Error (RLWE) problem, which depends on the ring dimension N and the total modulus size $|Q|$. The modulus size is computed as

$$|Q| = \sum_{i=0}^L |q_i|,$$

where each $|q_i|$ is the bit size of each modulus q_i , which are user-specified in CKKS implementations like TenSEAL. Larger N increases security by expanding the RLWE lattice dimension, while larger Q decreases security by reducing the effective noise in the RLWE problem. These security parameters have an inverse relationship with the overall security level.

The Homomorphic Encryption Standard provides upper bounds on $|Q|$ for each N at different security levels. These upper bounds are summarized in the following tables:

Table 4.4: Maximum modulus chain sizes for 128-bit security.

modulus polynomial size	maximum chain size
8192	218
16384	438
32768	881

Using these tables, the tool assigns a maximum security level to a given parameter set.

Table 4.5: Maximum modulus chain sizes for 192-bit security.

modulus polynomial size	maximum chain size
8192	165
16384	330
32768	660

Table 4.6: Maximum modulus chain sizes for 256-bit security.

modulus polynomial size	maximum chain size
8192	110
16384	220
32768	440

Let $S_\nu(N)$ denote the maximum chain size of Q for ring dimension N at security level $\nu \in \{128, 192, 256\}$. Then, for a given (N, Q) tuple, the security level is defined as

$$\text{sec}(N, Q) = \max\{\nu : |Q| < S_\nu(N)\}.$$

This lookup-based approach is standard in CKKS libraries, including Microsoft SEAL, as it avoids the computational cost of running a full lattice-attack estimators for each candidate parameter set.

4.4.2.2 MEMORY

The memory metric reflects the storage cost of ciphertexts, keys, and auxiliary structures. Similarly to the security, the memory usage associated with a parameter set is dependent on the tuple (N, Q) . Since all ciphertexts, keys, and auxiliary structures in CKKS are represented as polynomials in RNS form, we can estimate the size of a single RNS polynomial. For each RNS prime q_i , a polynomial requires

$$N \cdot q_i \cdot c$$

bytes, where c is the number of bytes needed to store a single coefficient. Summing over all primes yields the size of a single polynomial, and then we can assign factors of

it to various structures associated with CKKS. This assignment of the polynomial P is summarized in Table 4.7.

Table 4.7: Estimated memory requirements of CKKS components.

CKKS component	Memory size (bytes)
context	P
secret key	P
public key	$2P$
relinearization keys	$2PL$
ciphertext	$2P$

The total estimated memory footprint of a CKKS parameter set is defined as:

$$\text{mem}(N, \Delta, Q) = (\text{sum of all component sizes})$$

4.4.3 FINAL EVALUATION VECTOR

For each candidate parameter set, the tool returns an evaluation tuple

$$\text{eval}(N, \Delta, Q) = (\text{err}(N, \Delta, Q), \text{time}(N, \Delta, Q), \text{sec}(N, Q), \text{mem}(N, \Delta, Q)).$$

Each component captures a different aspect of performance for the given parameter set. The error metric $\text{err}(N, \Delta, Q)$ measures the numerical discrepancy between encrypted and plaintext inference, reflecting the accuracy of the encrypted computation. The runtime metric $\text{time}(N, \Delta, Q)$ quantifies the computational cost of performing encrypted evaluation. The security level $\text{sec}(N, Q)$ captures the cryptographic strength of the parameter set. Finally, the memory usage $\text{mem}(N, \Delta, Q)$ reflects the storage overhead associated with ciphertext representations in a public-key encryption system.

Together, these quantities define a multi-objective trade-off where improving one metric negatively impacts another. The tool therefore does not optimize a single metric in isolation, but identifies parameter sets that balance all four criteria.

4.5 PARAMETER RECOMMENDATION

In the parameter evaluation phase, the tool assigns a 4-tuple metric score to each candidate parameter set produced during search space generation. These associated empirical and analytical metrics are used to compare parameter sets and identify those that offer the best tradeoffs between correctness, efficiency, and security.

Because of the dimensionality of the evaluation metrics, no single parameter set is universally optimal. In particular, the empirical metrics (runtime and precision error) have an inverse relationship, as increasing the scale Δ may improve precision but also increase computational cost, while reducing the modulus chain may improve runtime but degrade accuracy. For this reason, the tool presents the results as a multi-objective optimization analysis rather than attempting to minimize a single scalar objective.

The tool first partitions all evaluated parameter sets by their assigned security level $v \in \{128, 192, 256\}$. Optimization is then performed independently within each security level, ensuring that the recommended parameter set for a given tier satisfies the associated security guarantees.

Within each level, the tool only considers the empirical metrics of runtime and precision error as the optimization objectives. The analytical metrics (security and memory) are used to filter and label parameter sets.

4.5.1 PARETO FRONTIER CONSTRUCTION

At each security level, the tool determines the set of optimal parameter sets along the two empirical metrics using a multi-objective optimization technique called a Pareto frontier. The Pareto frontier acts as a boundary distinguishing acceptable runtime-error compromises from suboptimal choices, ensuring that no parameter in the set can be improved without making at least one objective worse.

A Pareto frontier is determined through a “dominates” operation between parameter sets. Each parameter set is assigned a metric tuple

$$m_i = (r_i, e_i),$$

where r_i is runtime and e_i is error. A parameter set m_i “dominates” another set m_j if it is no worse in both metrics and strictly better in at least one:

$$m_i \prec m_j \iff \begin{cases} r_i \leq r_j, \\ e_i \leq e_j, \\ \text{and at least one inequality is strict.} \end{cases}$$

The Pareto frontier $\mathcal{P}$ consists of all the surviving points, i.e., all the points that are not dominated by another,

$$\mathcal{P} = \{m_i : \nexists m_j, m_j \prec m_i\}.$$

Any point not in $\mathcal{P}$ is suboptimal because another point exists that has both lower runtime and lower error. The frontier therefore represents the set of admissible tradeoffs between precision and efficiency for a fixed security level.

4.5.2 SELECTING THE KNEE POINT

Although the Pareto frontier captures all optimal tradeoffs, the tool, for streamlined usability, ultimately recommends a single parameter set per security level. This is accomplished by identifying the “knee” point on each Pareto frontier that achieves a good balance between runtime and error.

The knee point is computed by first normalizing both metrics:

$$\hat{r}_i = \frac{r_i - r_{\min}}{r_{\max} - r_{\min}}, \quad \hat{e}_i = \frac{e_i - e_{\min}}{e_{\max} - e_{\min}}.$$

Each frontier point is then assigned a combined score

$$s_i = \hat{r}_i + \hat{e}_i.$$

The recommended parameter set is the one minimizing s_i . This corresponds to the point closest to the idealized origin $(0,0)$ in normalized runtime-error space.

An illustration of this optimization and recommendation process is shown in Figure 4.2, in which each point represents an evaluated candidate parameter set and includes: dominated points (gray), frontier points (black), and the knee point (red).

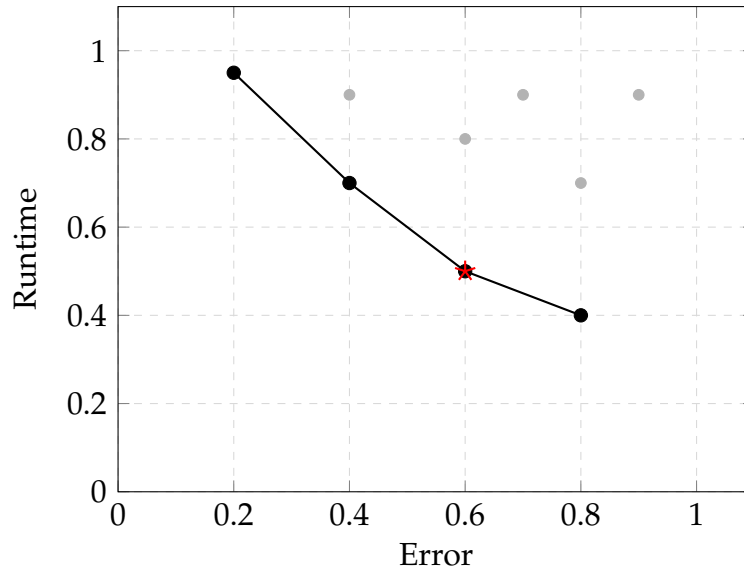


Figure 4.2: Illustration of a Pareto frontier showing dominated, non-dominated, and knee points.

4.5.3 FINAL INFERENCE EVALUATION

After the knee point is selected for each security level, the tool performs a final evaluation stage to validate the recommended parameter sets under realistic inference conditions. Unlike the earlier evaluation, which uses synthetic test vectors, this stage uses actual

entries from the original client dataset. It is important to note that this final evaluation would not be performed in real applications, as it involves sending plaintext information to the model. This step is meant to demonstrate the efficacy of the final recommended parameter sets.

The tool first randomly selects n entries from the dataset. For each recommended parameter set, the n entries are then sent through the model as both plaintext and ciphertext (encrypted using the input parameter set).

The tool computes two empirical inference metrics: error and runtime:

$$\text{err}_{\text{inf}}(N, \Delta, Q) = \max_{\vec{x}} \{ \|\text{Dec}(f(\text{Enc}(\vec{x}))) - f(\vec{x})\|_1 \},$$

$$\text{time}_{\text{inf}}(N, \Delta, Q) = t_{\text{end}} - t_{\text{start}}$$

The final inference evaluation tuple is

$$\text{eval}_{\text{inf}}(N, \Delta, Q) = (\text{err}_{\text{inf}}(N, \Delta, Q), \text{time}_{\text{inf}}(N, \Delta, Q)).$$

This final validation step ensures that the recommended parameters not only perform well on synthetic test vectors but also behave reliably on real client data.

4.6 OUTPUT

The tool produces a collection of outputs that fully document the experiment, its configuration, and the results of the parameter-selection process. These outputs are separated into two categories: (1) textual output printed to the terminal, and (2) file artifacts saved to an experiment directory.

4.6.1 TERMINAL OUTPUT

The terminal output summarizes the recommended parameter sets. There are at most three sets printed, one for each security level (128, 192, 256), as it could be the case that the search space did not generate any parameter sets satisfying a given level's constraints.

For each recommended parameter set, the tool prints:

- the polynomial degree N ,
- the scale Δ ,
- the modulus chain Q ,
- the assigned security level,
- the estimated memory usage of each CKKS component,
- the final inference metrics (error and runtime) computed on real client data.

4.6.2 FILE OUTPUT

All experiment artifacts are saved in a dedicated directory, ensuring reproducibility and enabling downstream analysis. The saved files include a CSV file that serves as a complete record of the search space and the gathered metrics. Each row corresponds to a candidate parameter set (N, Δ, Q) , and the columns are described in Table 4.8.

Additionally, the file output contains a JSON that stores the recommended parameter sets, including all information printed to the terminal. This serves as a machine-readable summary of the final selections. A second JSON file records the configuration settings used to run the experiment, including the number of samples for inference evaluation and test vector generation, as well as random seeds.

Finally, the output file includes a folder of PNG plots generated by the tool. It includes a plot for each valid security level, plotting runtime vs error with each point representing

Table 4.8: Description of output CSV file columns.

Column	Description
poly_degree	Polynomial modulus degree N
scale	Scaling factor Δ
chain	Coefficient modulus Q
chain_sum	Total modulus size $\sum q_i$
depth	Multiplicative depth $L + 1$
max_err	Error metric $\text{err}(N, \Delta, Q)$
runtime	Runtime metric $\text{time}(N, \Delta, Q)$
security	Security metric $\text{sec}(N, Q)$
total_memory	Memory metric $\text{mem}(N, \Delta, Q)$
status	Backend execution status (success/failure)
failure_reason	Error message if execution fails

a different candidate parameter set generated in the search space. The plots outline the points of the Pareto frontier in black, and mark the single recommended knee point with an X. The combined plot overlays all of the security levels onto a single plot for comparison.

CHAPTER 5

EVALUATION

This chapter aims to evaluate the effectiveness and practical usability of the presented CKKS parameter recommendation tool. Because selecting appropriate CKKS parameters involves non-trivial tradeoffs between security, runtime performance, and numerical precision, the tool presents a collection of close-to-optimal solutions. The experiments in this chapter assess whether the proposed tool can efficiently and effectively explore the CKKS parameter space and provide accurate recommendations for practical machine learning inference tasks. Specifically, the evaluation addresses four primary metrics:

Runtime efficiency How does the runtime of the parameter recommendation tool scale with increasing circuit depth?

Evaluation methodology Can representative test vectors accurately approximate the precision behavior of real dataset entries during encrypted inference evaluation of candidate parameter sets?

Parameter search efficiency Does the tool effectively explore the CKKS parameter space to produce recommended parameter configuration while avoiding the computational cost of an exhaustive search?

CKKS parameter tradeoffs What insights about runtime and precision tradeoffs can be obtained using the tool across different circuit depths and security levels?

To answer these questions, I performed a series of experiments using synthetic circuits, as well as real-world machine learning datasets and models. The results demonstrate that the tool provides reliable parameter recommendations, while also significantly reducing the computational cost of CKKS parameter selection.

5.1 EXPERIMENTAL SETUP

This section describes the datasets and models used in the experiments throughout this chapter. All experiments were performed on a personal laptop with a 12th Gen Intel Core i7, 16GB of RAM, and running a Windows Subsystem for Linux. The tool was implemented in Python, and the backend encryption library used was TenSEAL.

5.1.1 DATASETS

Two publicly available datasets were used to evaluate the parameter recommendation tool, which are both functionally available through the tool.

- **Diabetes Dataset:** a real-valued regression dataset consisting of 442 patient records, each described by 10 numerical features [13]. The features have varying distributions, such as for age, body mass index, blood pressure, and blood serum measurements.
- **Wisconsin Breast Cancer Dataset:** a binary classification dataset containing 569 instances, each described by 30 numerical features computed from digitized images of breast masses [34]. The features vary in magnitude as they capture different geometric and textural properties such as radius, perimeter, area, and smoothness.

5.1.2 MODELS

There are two types of models that were used during evaluation:

5.1.2.1 SYNTHETIC POLYNOMIAL MODEL

To analyze how circuit depth affects runtime and precision, I constructed a synthetic polynomial circuit with a multiplicative depth that can be explicitly controlled. The model is defined as

$$f(\vec{x}) = W\vec{x} + (W\vec{x})^{d+1},$$

where W is a randomly generated plaintext matrix sampled from a normal distribution. This construction ensures a total of d ciphertext-ciphertext multiplications and therefore a multiplicative depth d .

By varying d , I can systematically examine how an increase in circuit complexity impacts the parameter recommendation process with respect to the 4 evaluation metrics described previously.

5.1.2.2 DATASET-TRAINED MODELS

For the case study experiments, I used the tool's built-in training split functionality to train models directly on the datasets. These models represent realistic circuit workloads in encrypted machine learning settings.

5.2 RUNTIME EFFICIENCY

An important requirement for the practical deployment of the parameter recommendation tool is that it remains computationally efficient as the complexity of the evaluated circuit increases. In homomorphic encryption, the computational cost is strongly influenced by the multiplicative depth of the circuit, since deeper circuits require longer modulus chains, more rescaling procedures, and overall larger ciphertext parameters to maintain security. To evaluate the scalability of the tool, I measured how its runtime changes as the depth of the evaluated circuit increases. Here, the tool runtime refers to the time it took

to generate the candidate parameter space, evaluate each set, and produce recommended configurations.

For this experiment, the Diabetes dataset is used as the underlying input data. Additionally, the synthetic polynomial model is employed in order to vary the depth parameter in the range $1 \leq d \leq 6$ to measure the total runtime of the parameter recommendation tool.

The upper bound of $L = 6$ is chosen to reflect a practical region for CKKS evaluation that keeps the total experimental runtime manageable on the hardware used. Since the cost of homomorphic operations grows rapidly with circuit depth, extending beyond this range leads to disproportionately large runtimes. Figure 5.1 shows the observed total runtime as a function of circuit depth.

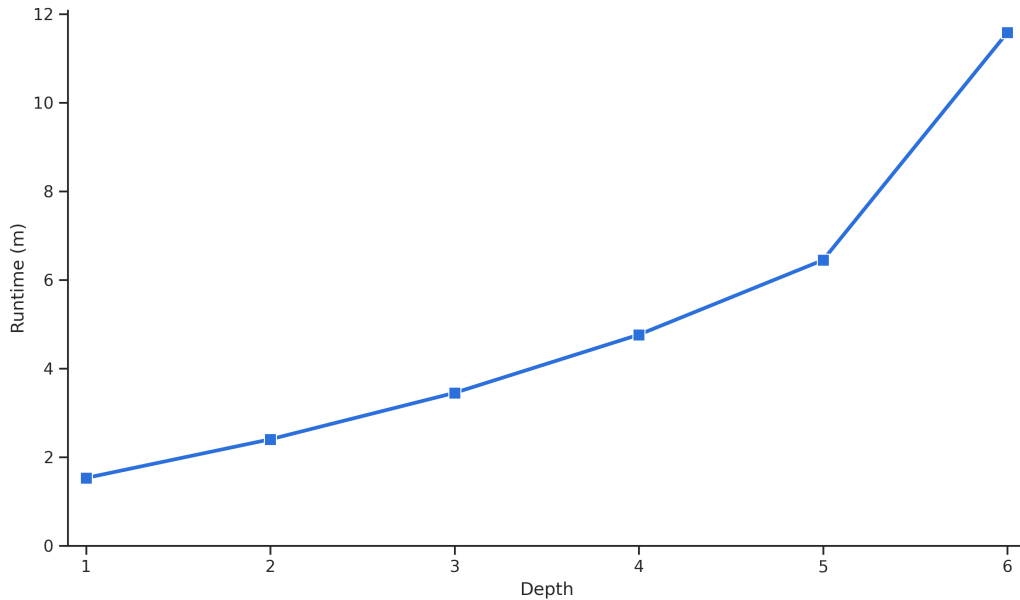


Figure 5.1: Tool runtime as a function of multiplicative depth.

The results demonstrate that the runtime of the tool increases at an increasing rate with circuit depth, reflecting the increase in computational cost of encrypted evaluation. The observed growth follows a quadratic-like rate, with a more significant increase from a depth of 5 to 6.

5.3 TEST VECTOR GENERATION STRATEGY

A central design component of the parameter recommendation tool is the use of representative test vectors to estimate precision loss during encrypted inference. The tool generates a set of test vectors intended to approximate the statistical properties of the dataset while preserving the privacy of actual entries. This section evaluates whether these representative vectors provide an accurate approximation of the precision behavior observed during real inference, focusing on two aspects of the test vector methodology: the effect of the sampling distribution and the effect of the number of test vectors used during candidate parameter evaluation. For the experiments, the underlying dataset used was the Diabetes dataset, with the model as the associated dataset-trained model.

5.3.1 SAMPLING STRATEGY

We first evaluate how the choice of sampling distribution affects the accuracy of the parameter evaluation phase to determine which distribution most accurately approximates the error observed during encrypted inference on real dataset entries.

For this experiment, the tool is executed on the same dataset using different sampling strategies for generating test vectors: uniform, Gaussian, truncated normal, and Student's t-distribution. For each sampling strategy, the tool produces recommended parameter sets across all three security levels.

For a given recommended parameter set, the relative error is computed between errors of the two evaluation phases: the ciphertext-plaintext evaluation error measured on representative test vectors, and the ciphertext-plaintext error measured on real dataset entries during the final evaluation phase. Specifically,

$$\frac{|E_{entry} - E_{test}|}{E_{entry}},$$

where E_{entry} is the error observed during inference on real dataset entries, and E_{test} is the error observed using synthetic test vectors.

This quantity measures how accurately the test-vector-based evaluation predicts the true error in practice. A value close to zero indicates strong agreement between the two phases, while larger values indicate that the test vectors provide a less reliable estimate of real-world performance. For example, a relative error of 0.3 means that the test-vector-based error differs from the true error by 30% of the true error magnitude, which is often acceptable as the goal is to capture overall error behavior under the intrinsically approximate CKKS scheme.

Figure 5.2 presents the distribution of relative errors for each sampling strategy across security levels, where lower relative errors indicate better performance.

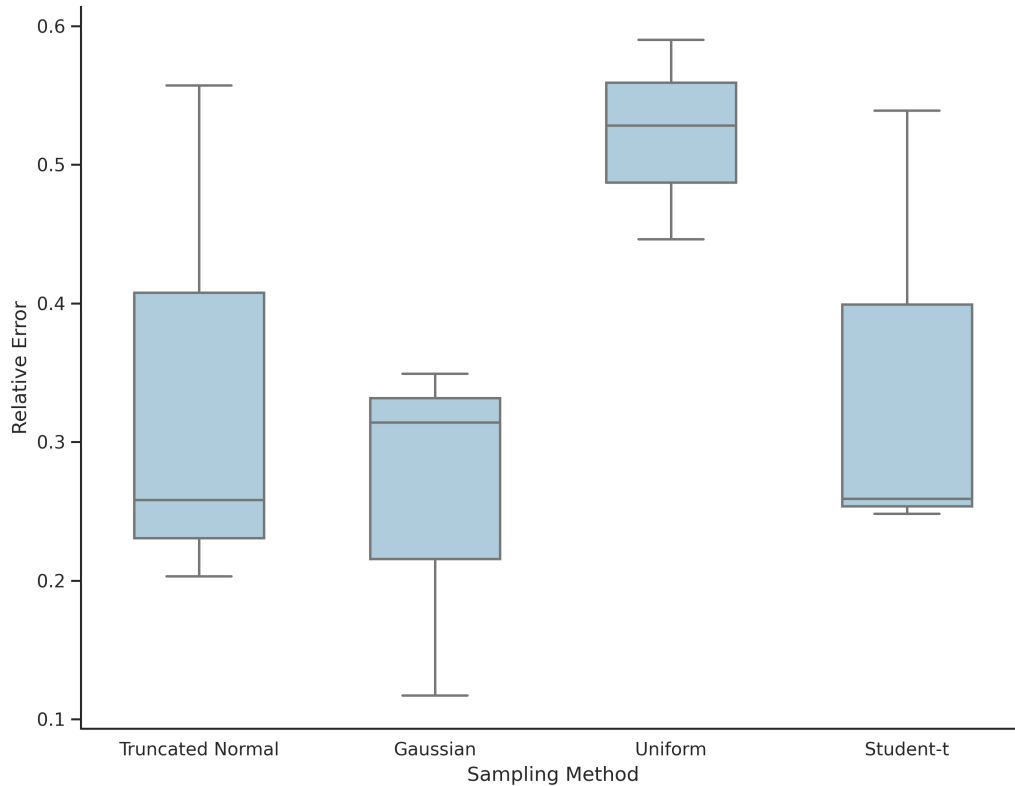


Figure 5.2: Accuracy of test vector sampling strategies

The results indicate that the sampling distribution influences both the stability and accuracy of the error estimation. Truncated normal sampling yields the lowest median

error but exhibits the greatest variability. Gaussian sampling achieves the lowest observed error values while maintaining a tighter upper bound. Uniform sampling produces a relatively narrow range of errors, but induces overall higher relative error values. Student's t-distribution had a tighter lower error bound but still contained a higher relative error value.

Overall, Gaussian sampling provided the most advantageous balance between accuracy and stability, making it a suitable choice for generating representative test vectors that reflect real inference precision.

5.3.2 NUMBER OF TEST VECTORS

In addition to the sampling distribution, the number of test vectors used during parameter evaluation can affect the accuracy of the estimated inference error. Using too few vectors may lead to unstable error estimates, while using too many vectors increases the computational cost of the evaluation phase, and can potentially jeopardize any safety provided by using statistically representative vectors instead of actual entries.

To examine this tradeoff, I varied the number of Gaussian-generated representative test vectors used during parameter evaluation and measured the relative error between the test vector error and the real dataset error observed during inference for each recommended parameter set. Figure 5.3 shows the median relative error as a function of the number of test vectors.

As the number of test vectors increases, the relative error gradually stabilizes as the oscillations decrease in magnitude. The error converges around a stable value once approximately 45 test vectors are used during the candidate parameter evaluation phase. Consequently, the parameter recommendation tool can reduce computational cost during parameter evaluation by minimizing the number of test vectors used, which will still maintain accurate precision estimates.

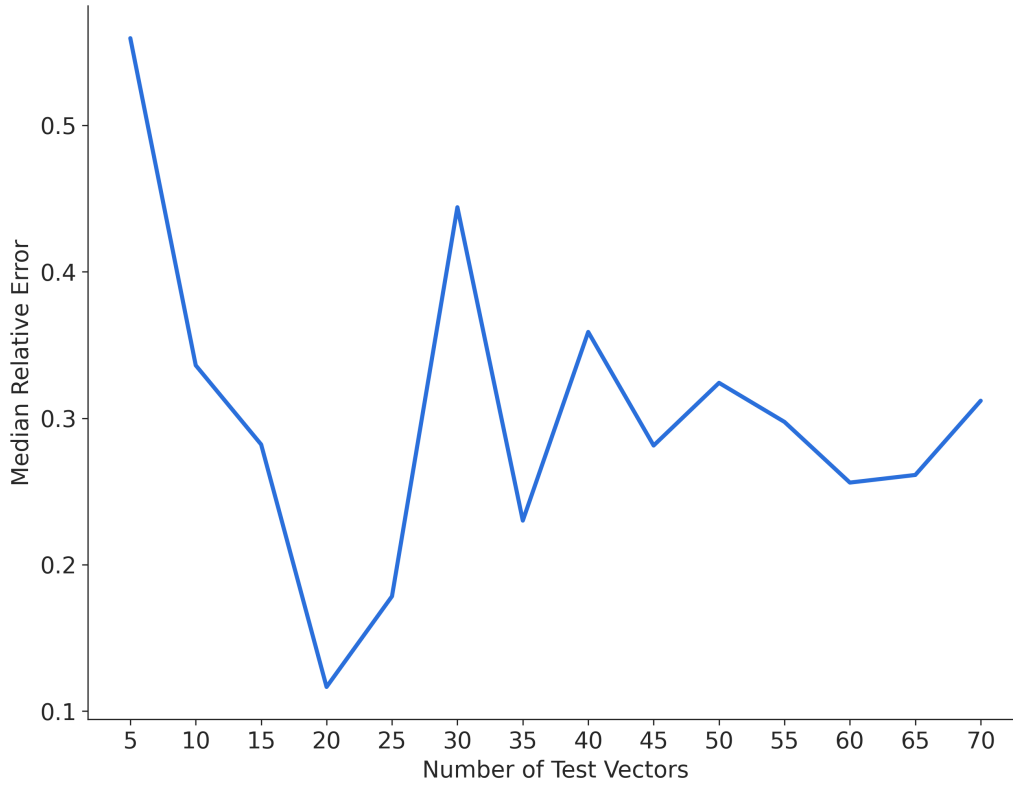


Figure 5.3: Accuracy comparison of test vector sampling strategies.

5.3.3 COMPARISON WITH RANDOM DATASET SAMPLING

To further evaluate the effectiveness of representative test vectors, I compared them against a baseline strategy that simply selects random entries directly from the dataset. This experiment consisted of two test vector generation strategies: Gaussian-generated representative vectors and random dataset entries.

For each strategy, I varied the number of test vectors used during parameter evaluation and computed the relative error between the test-vector inference error and the dataset entry inference error. This was calculated for every security level of the recommended parameter sets. Figure 5.4 shows the convergence behavior of the two strategies.

The results show that random sampling produces slightly lower error when a small number of test vectors are used. However, as the number of vectors increases, both

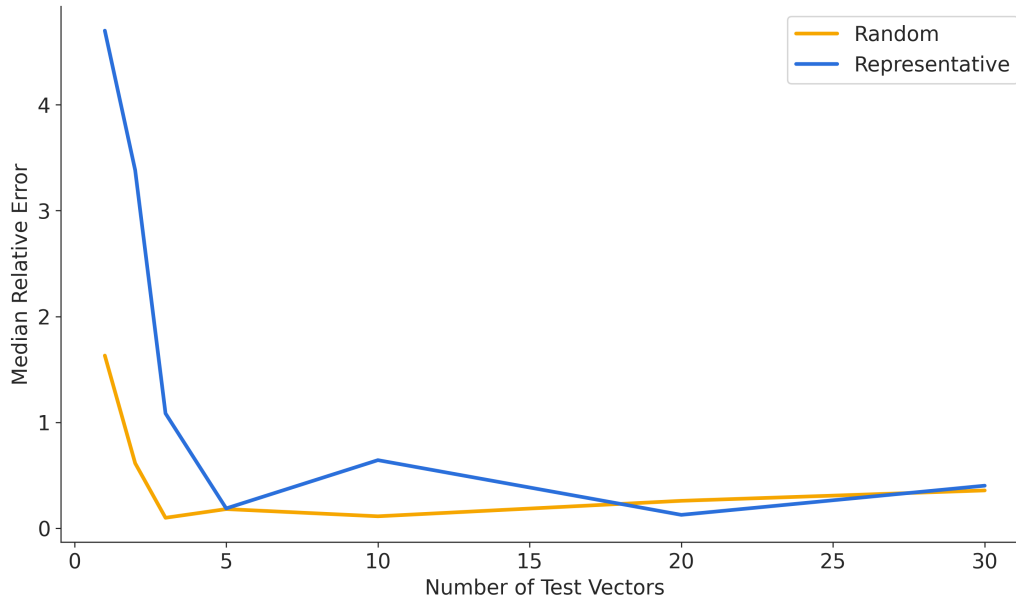


Figure 5.4: Error convergence of test vector generation strategies.

strategies converge to nearly identical error values when more than 10 test vectors are used.

The convergence behavior indicates that representative vectors provide an effective approximation of the dataset behavior when evaluating candidate parameters. While random sampling may provide a marginally more accurate estimate in low-sample contexts, it requires repeated access to the underlying dataset. In contrast, representative vectors are generated solely from summary statistics, which avoids the direct use of raw data. This distinction is particularly important in privacy-sensitive settings, where a central objective is to minimize data exposure. Furthermore, generating representative vectors is computationally more viable as the process does not scale with dataset size, making it a more practical approach for large datasets.

5.4 PARAMETER SEARCH SPACE EFFICIENCY

A central challenge in CKKS parameter selection is the size of the parameter search space, as parameter configurations vary across multiple dimensions. Since each evaluation

requires executing encrypted inference, even a small subset of the search space can take several minutes, especially when the depth of the underlying circuit is large. Thus, an exhaustive evaluation of all possible candidate combinations is most often practically nonviable for a client.

The parameter recommendation tool addresses this issue by constructing a restricted search space that focuses on parameter configurations that reflect the underlying dataset and model workload. To evaluate the effectiveness of this strategy, I compared the tool’s search space with a more exhaustive search over the parameter space. The experiment used the Diabetes dataset and its associated linear model as the underlying encrypted machine learning workload.

The baseline experiment was a near-exhaustive search over candidate parameter configurations, which serves as a practical approximation of a fully exhaustive one while remaining computationally viable. The search space is constructed by modifying the default search space generation method, increasing the number of possible choices for: the scale Δ , the first modulus q_0 , and the modulus chain length L . All parameter configurations satisfy the minimum security constraints based on the Homomorphic Encryption Standard. This expanded search space represents a significantly larger set of candidate configurations than those explored by the tool.

The increase in both the number of candidate parameter sets evaluated and the total tool runtime is summarized in Table 5.1.

Table 5.1: Comparison of runtime and candidate parameter counts between tool-based and near-exhaustive search.

Search Space	Runtime (minutes)	Candidate Parameter Sets
Full	9.75	377
Tool	1.48	80

The tool evaluates almost 5 times fewer parameter sets, and it runs approximately 6.5 times faster. Although the tool significantly restricts evaluation to a much smaller number of candidate configurations, it is important to verify that the reduced search space still

captures the most relevant regions of the parameter space. Figure 5.5 overlays the plots produced by the tool and the near-exhaustive search, with the security metric collapsed for readability.

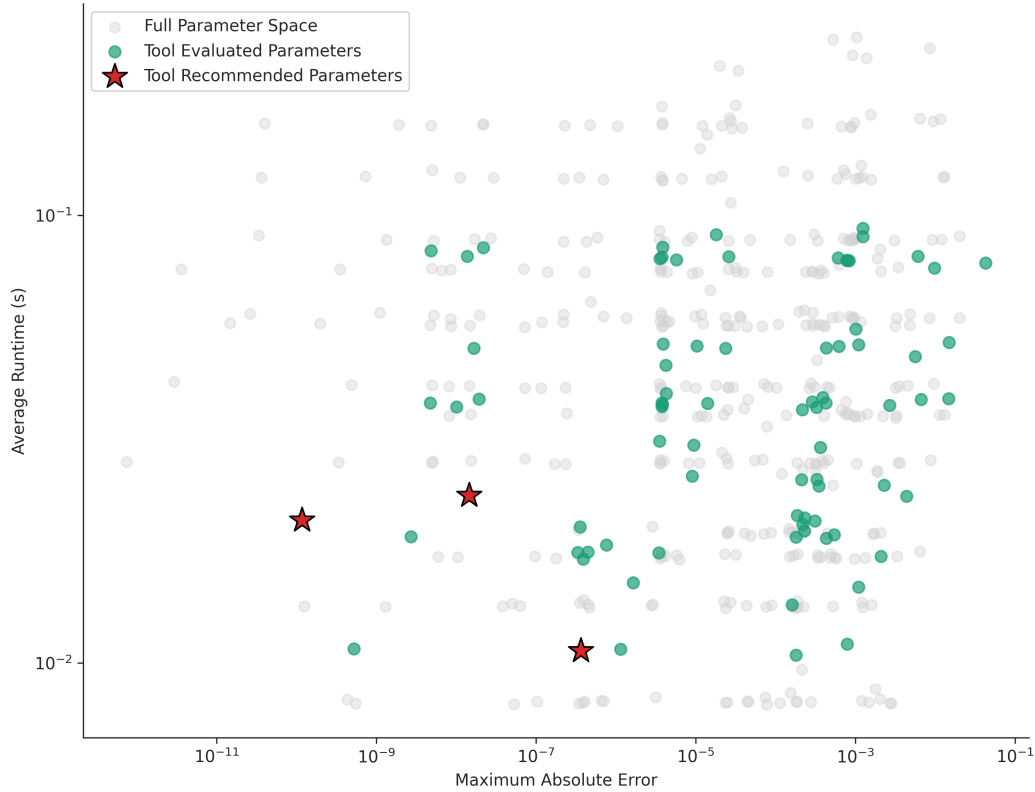


Figure 5.5: Comparison of full parameter search space and tool-based exploration.

The results show that the tool evaluates parameter configurations concentrated near the region of the parameter space where the optimal tradeoffs between runtime and precision occur. In particular, the recommended configurations identified by the tool lie close to the Pareto frontier obtained from the near-exhaustive search.

While the near-exhaustive search identified a small number of configurations that slightly extend beyond the tool’s Pareto frontier, these improvements are marginal and come at a significantly increased computational cost. This demonstrates that the tool’s default search space generation method provides a substantial reduction in runtime while still identifying high-quality parameter configurations. Moreover, when greater coverage

of the parameter space is required, the tool’s search procedure can be systematically expanded as demonstrated in this experiment.

5.5 CKKS PARAMETER TRADEOFFS

Beyond validating the parameter recommendation tool, it is also valuable to examine the broader CKKS parameter trends that emerge when applying the developed tool across varying circuit depths and security levels. In particular, I analyzed how circuit depth influences both the runtime of encrypted computation and the precision of decrypted results.

These experiments were conducted using the Diabetes dataset and the synthetic polynomial model. The synthetic model provides direct control over the multiplicative depth of the underlying computational circuit.

5.5.1 RUNTIME AND MULTIPLICATIVE DEPTH

In homomorphic encryption, circuit depth is one of the primary factors determining computational cost. Each additional ciphertext–ciphertext multiplication increases noise growth, consequently requiring additional rescaling operations, longer modulus chains, and overall larger ciphertext objects.

For each depth value, the parameter recommendation tool evaluates candidate parameter sets and measures the average runtime required to perform encrypted inference. For each security level, I computed the mean runtime across all evaluated candidate parameter sets. The resulting relationship between circuit depth and runtime is shown in Figure 5.6.

The results show that the average encrypted inference runtime increases at an increasing rate as circuit depth grows. Additionally, higher security levels require larger

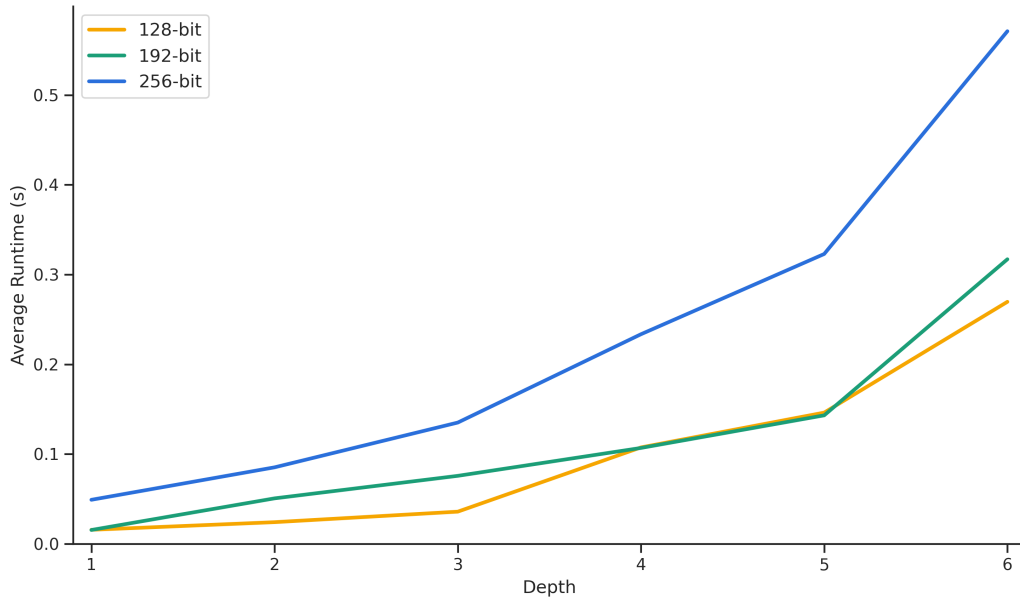


Figure 5.6: Encrypted inference runtime as a function of multiplicative depth (by security level). parameters to maintain their security guarantees as the circuit depth increases, which induces the clear stratification of the 256-bit security level.

5.5.2 PRECISION AND MULTIPLICATIVE DEPTH

In addition to runtime, circuit depth also influences the precision of decrypted outputs. Since CKKS supports approximate arithmetic, noise accumulates throughout the computation, especially during ciphertext–ciphertext multiplications and rescaling operations. To examine how depth affects precision, I measured the median error metric across the evaluated candidate parameter sets for each security level as a function of circuit depth. The resulting relationship between circuit depth and precision is shown in Figure 5.7.

The observed error remains more stable as the security level increases, with the 256-bit level having very minimal variation. As the security level decreases, the change in median error behaves more unsteadily with circuit depth, such as the 128-bit level having large jumps from 3 to 4 and then 4 to 5. Despite this more erratic behavior, the error trend

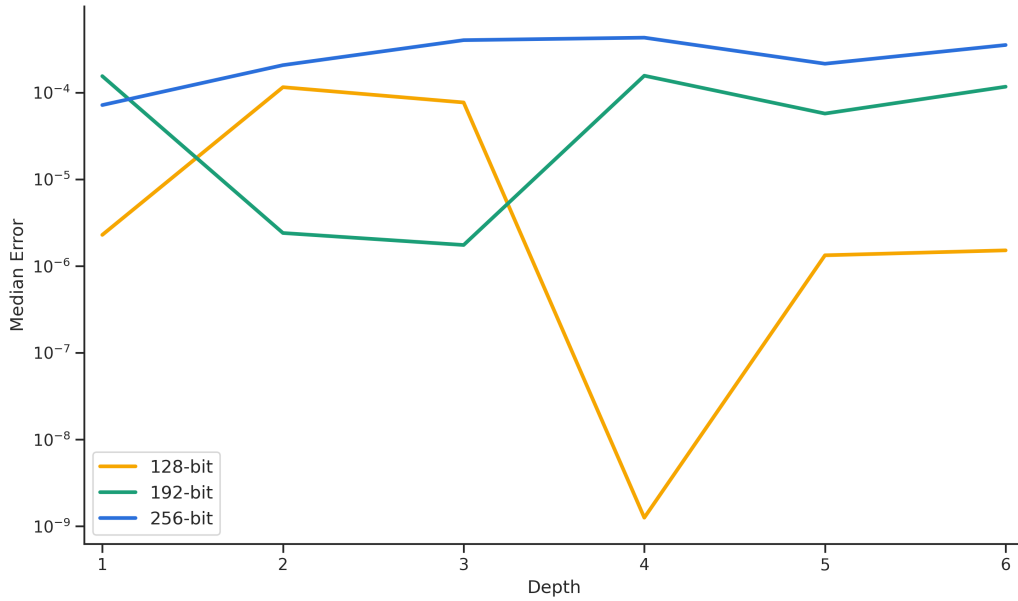


Figure 5.7: Encrypted inference precision as a function of multiplicative depth (by security level). across security levels remains close to horizontal, implying that depth has less of an impact on the precision of the computation.

Although the circuit depth increases, the underlying dataset remains the same, so the tool’s data-informed choice for the scale Δ remains the same. Thus, as depth increases, the tool not only generates longer modulus chains, but also generates consistent modulus chain sizes to account for Δ . This might require a larger N to maintain a given security level, but the precision remains largely unchanged across the tested depth range.

5.6 CASE STUDIES

To demonstrate the practical applicability of the parameter recommendation tool, I applied it directly to the two machine learning datasets and associated models discussed in this chapter. These case studies in the healthcare domain illustrate how the tool can identify parameter configurations that balance precision and runtime in practical workload contexts.

For each dataset, the tool evaluates candidate CKKS parameter configurations across

multiple security levels and produces a set of recommended parameter sets that lie on the Pareto frontier of the tradeoffs between runtime and precision.

These experiments highlight the inherently multi-objective nature of CKKS parameter selection. Increasing parameter sizes can improve numerical precision (through the scale Δ) and security guarantees (through modulus degree N and the modulus chain Q). However, these improvements come at the cost of increased computational runtime. Conversely, selecting smaller parameters may improve performance but can lead to potentially insufficient precision during encrypted computation.

The resulting runtime-precision plots reveal a clear Pareto frontier among candidate configurations. Parameter sets located along this frontier represent optimal tradeoffs, where improving one metric would necessarily reduce another.

5.6.1 DIABETES DATASET

The tool is first evaluated using the Diabetes dataset and the associated trained linear regression model. The tool analyzes the model and the dataset's statistical properties to generate candidate CKKS parameter configurations capable of supporting encrypted inference.

Figures 5.8, 5.9, and 5.10 show the relationship between runtime and precision for evaluated parameter configurations at security levels of 128 bits, 192 bits, and 256 bits, respectively.

Points located near the lower-left region of the plots represent desirable parameter configurations, as they achieve low runtime while maintaining high precision. To provide a more comprehensive analysis, Figure 5.11 combines all security levels into a single visualization.

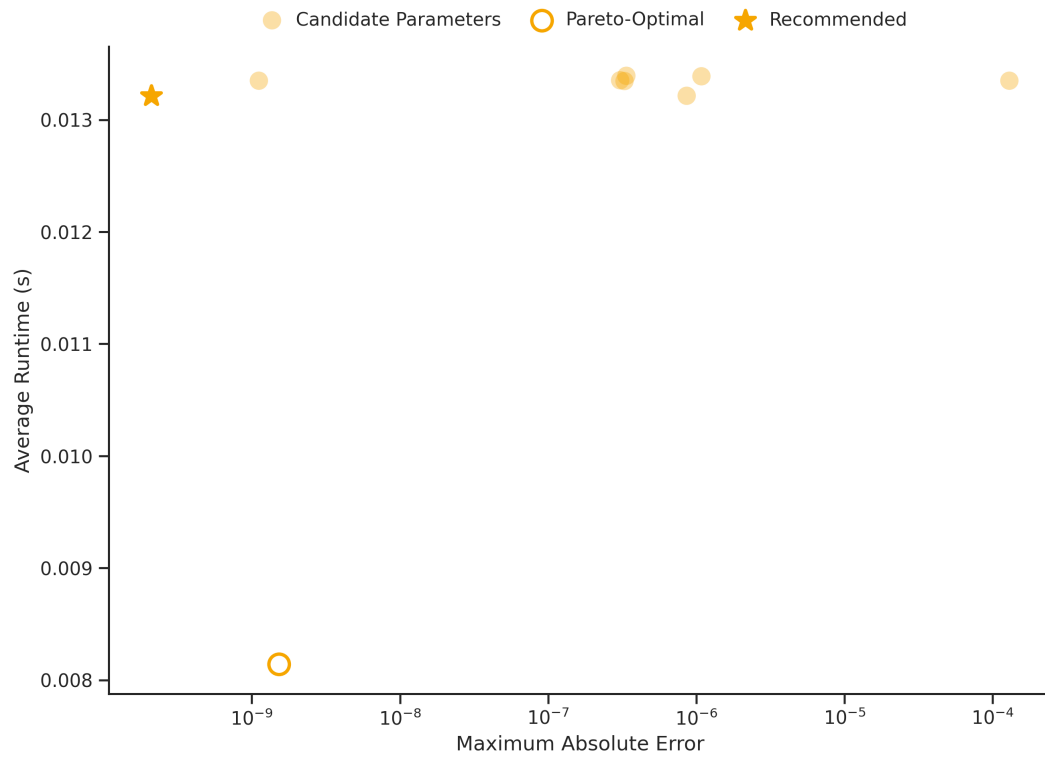


Figure 5.8: Precision–runtime tradeoff at 128-bit security level (Diabetes dataset).

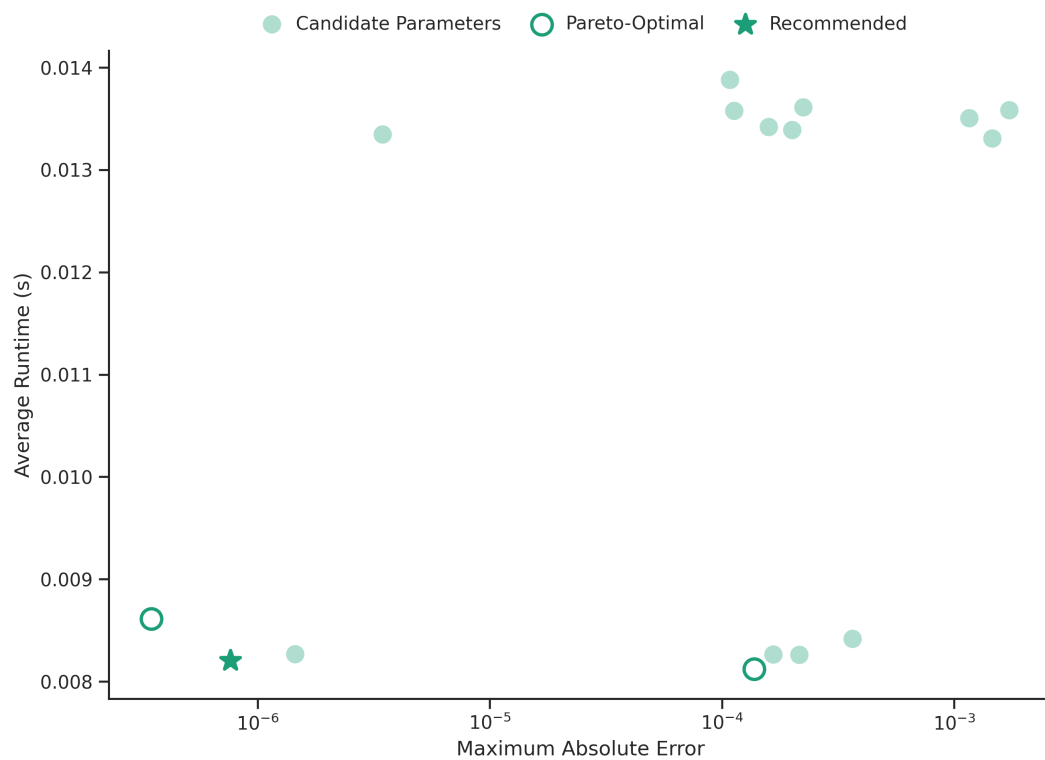


Figure 5.9: Precision–runtime tradeoff at 192-bit security level (Diabetes dataset).

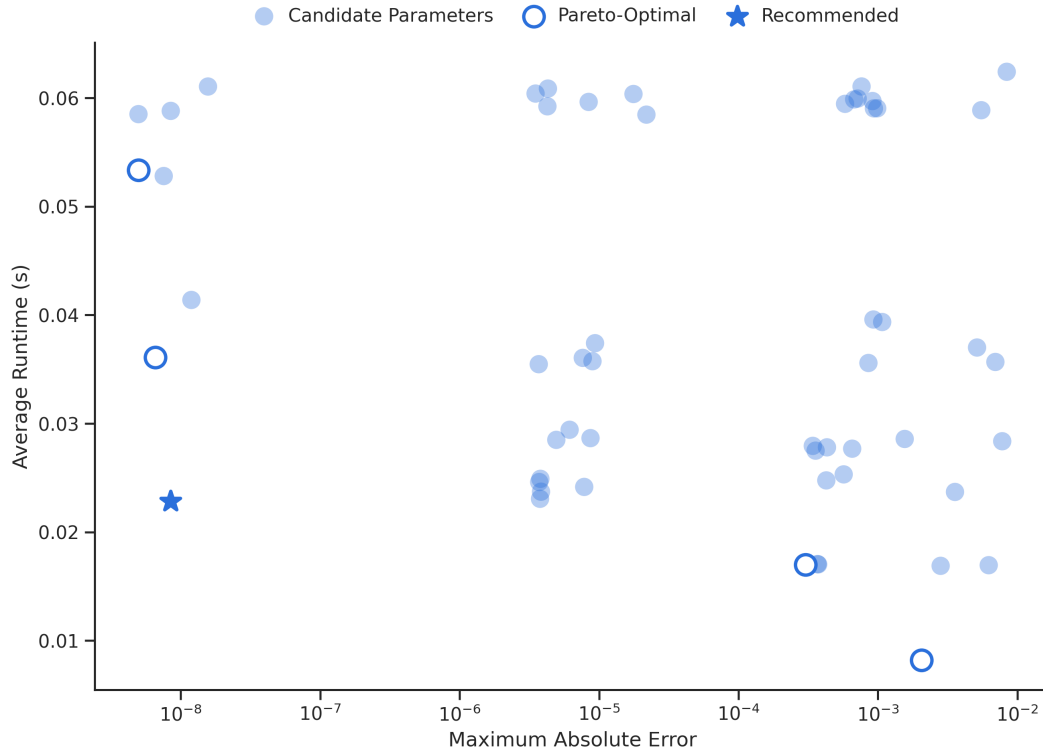


Figure 5.10: Precision–runtime tradeoff at 256-bit security level (Diabetes dataset).

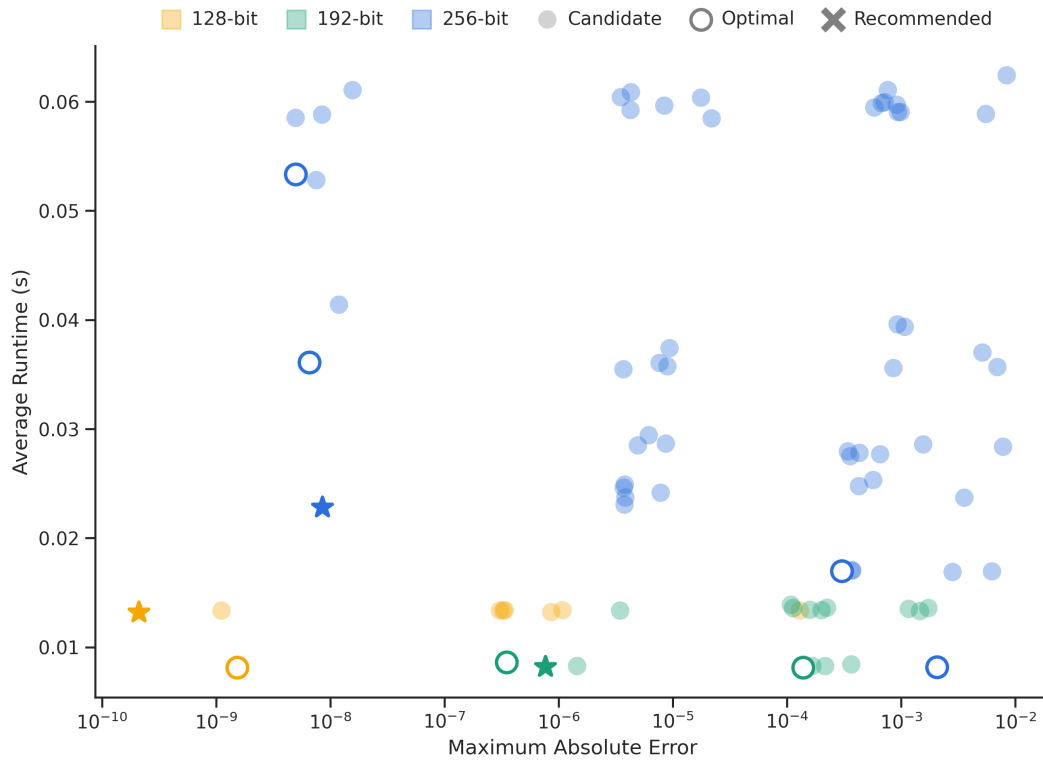


Figure 5.11: Precision–runtime tradeoff of all security levels (Diabetes dataset).

This aggregated plot highlights how higher security levels shift the optimal parameter frontier toward higher runtime due to the larger polynomial modulus degrees required to maintain security guarantees.

5.6.2 WISCONSIN BREAST CANCER DATASET

To further evaluate the generality of the tool, it is applied to the Wisconsin Breast Cancer dataset and its associated trained model. This workload represents a different encrypted machine learning task and dataset feature distribution, allowing the behavior of the parameter recommendation process across datasets to be tested.

Figures 5.12, 5.13, and 5.14 show the runtime-precision tradeoff across candidate parameter sets at security levels of 128 bits, 192 bits, and 256 bits, respectively. Figure 5.15 combines all security levels into a single visualization.

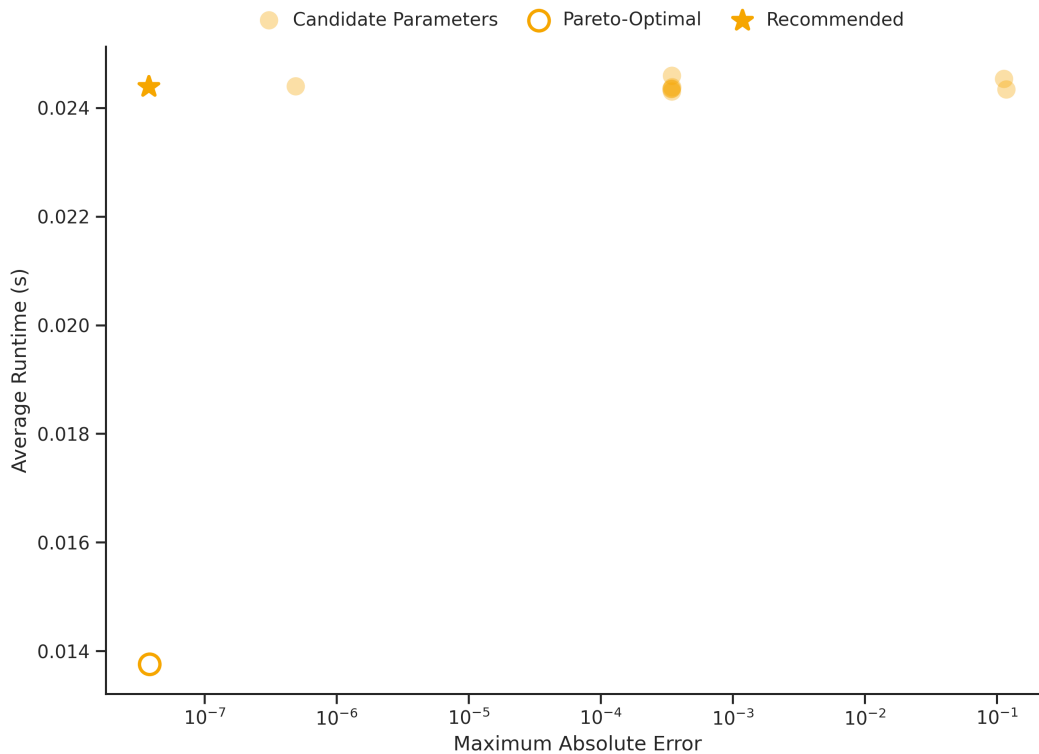


Figure 5.12: Precision–runtime tradeoff at 128-bit security level (Breast Cancer dataset).

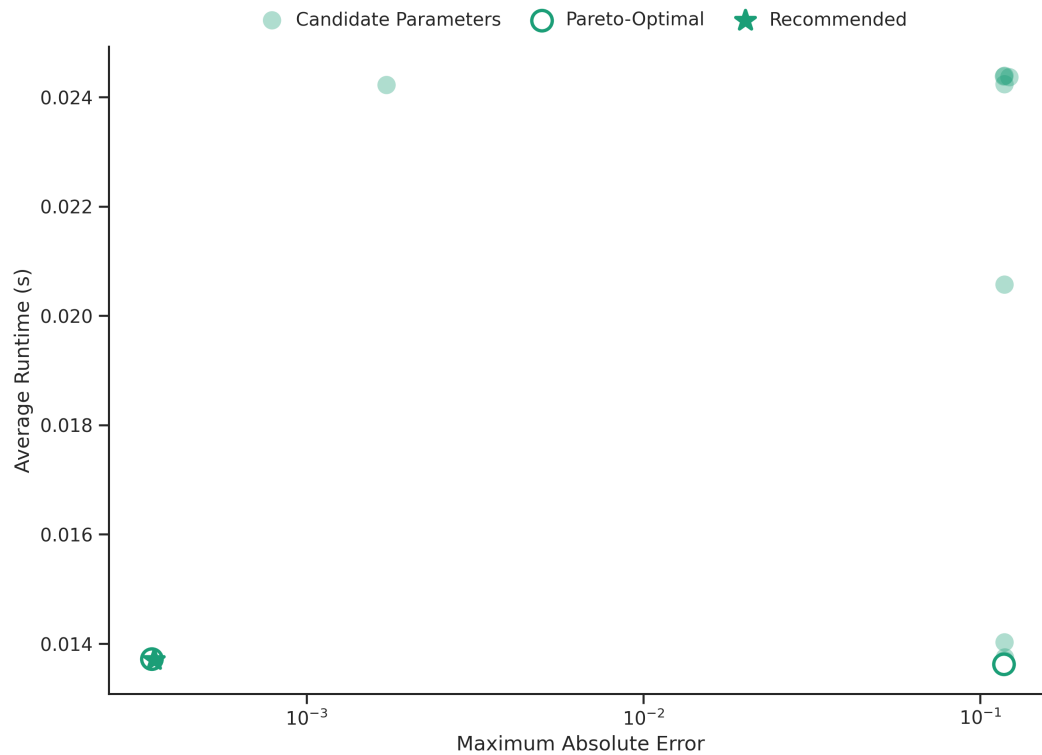


Figure 5.13: Precision–runtime tradeoff at 192-bit security level (Breast Cancer dataset).

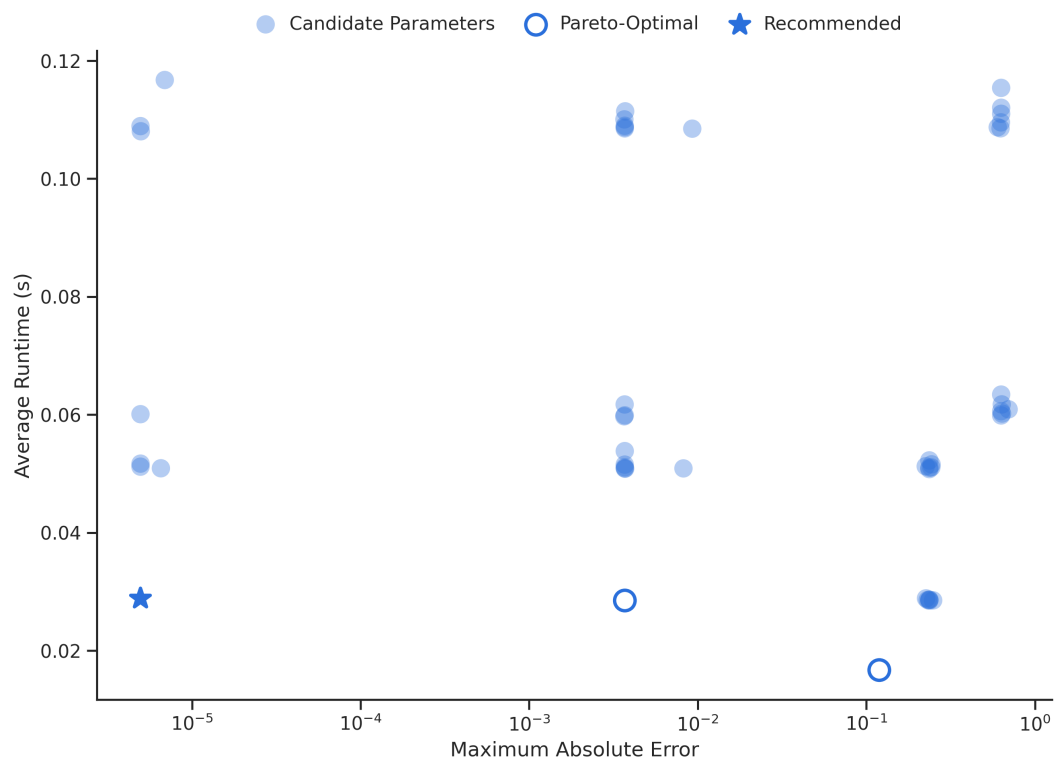


Figure 5.14: Precision–runtime tradeoff at 256-bit security level (Breast Cancer dataset).

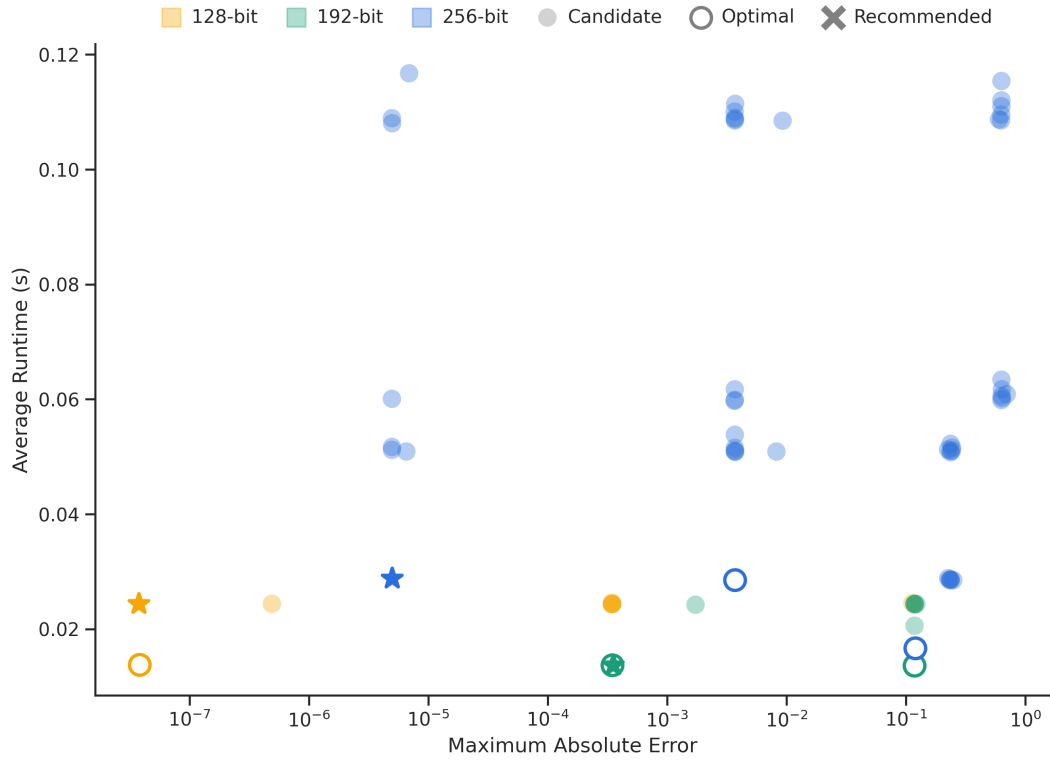


Figure 5.15: Precision–runtime tradeoff of all security levels (Breast Cancer dataset).

The resulting spread of parameter configurations follows a similar trend to that observed for the Diabetes dataset, with the recommended configurations identified by the tool lying on the Pareto frontier that represents optimal balances between precision and runtime.

CHAPTER 6

DISCUSSION

The experiments presented in Chapter 5 evaluated the effectiveness of the CKKS parameter recommendation tool across several aspects, namely runtime efficiency, accuracy of the test vector evaluation strategy, and parameter search efficiency. While the previous chapter focused on presenting and analyzing empirical results, this chapter interprets those findings to draw broader conclusions about parameter selection in CKKS-based systems.

In particular, this discussion highlights three key insights. First, the results demonstrate that representative test vectors and dataset-informed parameter search strategies provide a reliable foundation for CKKS parameter selection. Second, the results show that presenting parameter configurations as part of a comprehensive trade-off analysis, as opposed to a single recommended choice, can improve the practical usability of CKKS by enabling more informed decision-making. Finally, the evaluation results demonstrate the tool’s usefulness for workload-specific experimentation, enabling users to explore how parameter choices impact CKKS behavior under different conditions.

These findings inform how automated, data-driven tools can assist developers and researchers in navigating the complex trade-offs between security, runtime, and numerical precision in practical privacy-preserving machine learning systems.

6.1 DATA-DRIVEN PARAMETER SELECTION

Selecting appropriate parameters for CKKS-based computation is a complex task for non-experts, as each choice affects security, runtime performance, numerical precision, and computational costs. In practice, developers often rely on heuristic guidelines or manual experimentation when configuring homomorphic encryption systems. However, these approaches can lead to inefficient parameter choices or unnecessary computational overhead. The results of this work suggest that parameter selection can instead be grounded in the structure of the underlying dataset-model workload.

First, representative test vectors provide an effective proxy for dataset behavior when evaluating candidate parameter sets. By constructing these vectors from simple statistical summaries, the tool is able to effectively approximate the numerical behavior of encrypted inference without requiring repeated access to the original dataset. This approach reduces the need for costly or potentially sensitive data access during the tuning process.

Second, the tool’s search space generation heuristic utilizes the dataset to inform the choice of the scale Δ , specifically from magnitude and resolution analyses. Additionally, the multiplicative depth of the model is used to inform choices of the modulus chain Q . This approach replaces the need for an exhaustive parameter search to identify close-to-optimal parameter configurations, which significantly reduces the computational cost.

Together, these results indicate that CKKS parameter selection can be made more systematic and reliable by incorporating automated data-driven evaluation methods. Rather than relying on broad heuristics, parameter choices can be tailored to the numerical properties of the input data, leading to more efficient and application-specific encrypted machine learning systems.

6.2 TRADE-OFF ANALYSIS AND USABILITY

A key challenge in CKKS parameter selection is the inherently multi-objective nature of the problem. Improvements in precision or security often require larger parameter values, which increase computational cost. Conversely, reducing runtime typically involves smaller parameter choices that may compromise accuracy. As a result, there is no single parameter configuration that is universally optimal across all applications.

Existing approaches to CKKS parameterization rely on user-provided security and precision requirements to produce a single “best” parameter set. In contrast, the proposed tool improves the usability of CKKS-based systems for non-experts by automatically incorporating the underlying workload into parameter selection to produce a comprehensive overview of the multiple performance-optimal parameter configurations.

The case study evaluations in this thesis demonstrate that optimal parameter sets lie along a Pareto frontier that reflects the optimal tradeoffs between the competing objectives of runtime, security, and precision. By presenting the frontier, the proposed tool enables users to reason about the full span of achievable performances for a specific workload. This enables informed decisions based on general preferences, such as favoring lower latency, higher precision, or stronger security guarantees, but notably does not require pre-determined priorities.

This automatic and comprehensive approach is particularly valuable in real-world deployment scenarios, where application requirements vary, and non-experts must consider various trade-offs. Moreover, it reduces reliance on trial-and-error parameter tuning, which is both time-consuming, difficult to reproduce, and often unintuitive. By providing a structured view of the trade-off space, the tool supports more transparent and systematic decision-making, making homomorphic encryption more accessible to developers and organizations.

6.3 WORKLOAD-SPECIFIC PARAMETER EXPLORATION

In addition to its direct role in recommending parameter configurations, the proposed tool provides practical value as a means of exploring the behavior of CKKS encryption under different workload conditions. The systematic evaluation of candidate parameter sets allows users to observe how changes in parameters and evaluation methods influence runtime, security, and precision for a given model and dataset.

This capability is particularly useful in experimental and development settings. For example, varying properties such as circuit depth or input distributions allow users to better understand how these factors affect encrypted computation. The experiments presented in Chapter 5 showcase this experimental exploration, such as the effect of circuit depth and test vector sampling strategies on inference precision estimation.

As a result of the structured and reproducible evaluation of parameter sets, the tool can assist in both selecting parameters and in reasoning about design choices within practical settings. This makes it a useful resource for deploying encrypted machine learning systems and for studying CKKS performance characteristics.

6.4 FUTURE WORK

While the evaluation demonstrates the effectiveness and applicability of the proposed CKKS parameter recommendation tool, two central considerations should be taken into account when interpreting the results. These, in turn, highlight opportunities for future work aimed at extending the robustness of the proposed framework.

6.4.1 DATASET AND MODEL SCOPE

The experiments in this work were conducted using a toy polynomial model and a relatively small and well-behaved real-world dataset. The experiments provide a useful

baseline for evaluating the parameter recommendation tool and demonstrating a proof of concept.

However, models such as neural networks or models with higher multiplicative depth may exhibit different runtime and precision characteristics. Moreover, datasets with different distributions may impose different precision requirements for encrypted inference. Expanding the evaluation to include a wider variety of model architectures and datasets would provide further insight into the general applicability of the proposed framework. The results of such experiments could also help guide improvements for the dataset- and model-informed parameter generation heuristics.

6.4.2 REPRESENTATIVE TEST VECTOR SECURITY

The experimental results demonstrate that statistically representative test vectors can approximate the precision behavior of encrypted inference while avoiding the use of sensitive dataset entries. However, the security implications of using proxy inputs require further consideration.

In the proposed framework, representative test vectors are sent to the outsourced model for evaluation. Although these vectors are designed to reflect the statistical properties of the underlying dataset rather than revealing actual data points, they are still sent in plaintext. As a result, the approach assumes that revealing statistical properties of the dataset does not compromise the privacy requirements of the application.

Future work could investigate approaches for strengthening the privacy guarantees of the evaluation process. Potential directions include exploring alternative sampling strategies, minimizing the number of test vectors required for accurate evaluation, or integrating differential privacy to limit the information revealed about the underlying dataset.

That said, the experimental results presented in this thesis provide strong evidence that the proposed parameter recommendation framework offers a practical and effective

method for exploring the CKKS parameter space. By combining automated parameter generation, representative test vector evaluation, and comprehensive runtime-precision analysis, the tool provides a structured approach for identifying parameter configurations suitable for encrypted inference tasks.

CONCLUSION

While cloud-based machine learning models offer powerful predictive capabilities, their use requires users to share data with external systems, which limits their applicability in privacy-critical domains such as healthcare and finance. Homomorphic encryption, and in particular the CKKS scheme, provides a compelling solution by supporting computation directly on encrypted data.

However, the practical deployment of CKKS remains challenging due to the complexity of its parameterization. Selecting appropriate parameters requires balancing the competing objectives of security, numerical precision, and computational efficiency. This thesis addressed this challenge by proposing an automated and data-driven parameter recommendation framework for CKKS-based machine learning inference. The proposed tool contributes to this problem in three key ways.

First, it shows that incorporating dataset and model characteristics into parameter generation leads to an efficient and targeted exploration of the parameter space for the specific workload. The experimental results indicate that the full search space was significantly collapsed while still identifying parameter configurations that perform with optimal trade-offs.

Second, it demonstrates that representative test vectors can serve as an effective proxy for evaluating inference error, enabling parameter evaluation without requiring direct access to sensitive data. The experiments show that a sufficient number of test vectors

approximate the same as the direct usage of dataset entries, reinforcing the viability of employing dataset-proxy evaluation strategies in privacy-preserving settings.

Third, it presents parameter configurations as part of a trade-off analysis, allowing users to assess the full range of performance outcomes rather than relying on a single recommended solution. The automated and comprehensive process improves the usability of privacy-preserving machine learning by lowering the barrier to entry and enabling a more informed deployment in real-world systems.

Further experimentation across diverse datasets and application domains, such as finance, would provide additional insight into how the underlying data distributions influence the tool's performance. Research avenues such as this would help refine data-informed parameter construction and improve the design of representative test vectors for evaluating configurations. This, in turn, would strengthen the framework's practicality and robustness, as it is extended to broader real-world settings.

Privacy concerns will continue to shape the future of machine learning and artificial intelligence, increasing the need for tools that make advanced cryptographic techniques more accessible. By introducing an automated, data-driven approach to CKKS parameter selection, this work contributes to improving the practical usability of privacy-preserving machine learning systems.

REFERENCES

- [1] Martin Albrecht, Rachel Player, and Samuel Scott. “On the concrete hardness of Learning with Errors”. English. In: *Journal of Mathematical Cryptology* 9.3 (Oct. 2015), pp. 169–203. ISSN: 1862-2976. DOI: [10.1515/jmc-2015-0016](https://doi.org/10.1515/jmc-2015-0016) (page 34).
- [2] Martin Albrecht et al. *Homomorphic Encryption Security Standard*. Tech. rep. Toronto, Canada: HomomorphicEncryption.org, Nov. 2018 (pages 35, 40, 43–44).
- [3] Andreea Alexandru et al. *Application-Aware Approximate Homomorphic Encryption: Configuring FHE for Practical Use*. Cryptology ePrint Archive, Paper 2024/203. 2024. DOI: [10.62056/ayl83z10k](https://doi.org/10.62056/ayl83z10k). URL: <https://eprint.iacr.org/2024/203> (page 42).
- [4] Ahmad Al Badawi et al. *OpenFHE: Open-Source Fully Homomorphic Encryption Library*. Cryptology ePrint Archive, Paper 2022/915. 2022. URL: <https://eprint.iacr.org/2022/915> (pages 13, 35).
- [5] Ayoub Benaissa et al. *TenSEAL: A Library for Encrypted Tensor Operations Using Homomorphic Encryption*. 2021. arXiv: [2104.03152](https://arxiv.org/abs/2104.03152) [cs.CR]. URL: <https://arxiv.org/abs/2104.03152> (page 35).
- [6] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. “(Leveled) fully homomorphic encryption without bootstrapping”. In: *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*. ITCS ’12. Cambridge, Massachusetts: Association for Computing Machinery, 2012, pp. 309–325. ISBN: 9781450311151. DOI: [10.1145/2090236.2090262](https://doi.org/10.1145/2090236.2090262). URL: <https://doi.org/10.1145/2090236.2090262> (pages 8, 18).

- [7] José Cabrero-Holgueras and Sergio Pastrana. *Towards Automated Homomorphic Encryption Parameter Selection with Fuzzy Logic and Linear Programming*. 2023. arXiv: 2302.08930 [cs.CR]. URL: <https://arxiv.org/abs/2302.08930> (page 43).
- [8] Hao Chen, Kim Laine, and Rachel Player. *Simple Encrypted Arithmetic Library - SEAL v2.1*. Cryptology ePrint Archive, Paper 2017/224. 2017. URL: <https://eprint.iacr.org/2017/224> (page 35).
- [9] Jung Hee Cheon et al. “Homomorphic Encryption for Arithmetic of Approximate Numbers”. In: *Advances in Cryptology – ASIACRYPT 2017*. Ed. by Tsuyoshi Takagi and Thomas Peyrin. Cham: Springer International Publishing, 2017, pp. 409–437. ISBN: 978-3-319-70694-8 (pages 3, 9, 18).
- [10] Roshan Dathathri et al. “CHET: an optimizing compiler for fully-homomorphic neural-network inferencing”. In: *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. PLDI 2019. Phoenix, AZ, USA: Association for Computing Machinery, 2019, pp. 142–156. ISBN: 9781450367127. DOI: 10.1145/3314221.3314628. URL: <https://doi.org/10.1145/3314221.3314628> (page 41).
- [11] Roshan Dathathri et al. “EVA: an encrypted vector arithmetic language and compiler for efficient homomorphic computation”. In: *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, June 2020, pp. 546–561. DOI: 10.1145/3385412.3386023. URL: <http://dx.doi.org/10.1145/3385412.3386023> (page 41).
- [12] Austin Ebel, Karthik Garimella, and Brandon Reagen. “Orion: A Fully Homomorphic Encryption Framework for Deep Learning”. In: *ASPLOS ’25*. Rotterdam, Netherlands: Association for Computing Machinery, 2025, pp. 734–749. ISBN: 9798400710797. DOI: 10.1145/3676641.3716008. URL: <https://doi.org/10.1145/3676641.3716008> (page 41).

- [13] Bradley Efron et al. *Least Angle Regression*. 2004. URL: https://web.stanford.edu/~hastie/Papers/LARS/LeastAngle_2002.pdf (page 71).
- [14] Junfeng Fan and Frederik Vercauteren. *Somewhat Practical Fully Homomorphic Encryption*. Cryptology ePrint Archive, Paper 2012/144. 2012. URL: <https://eprint.iacr.org/2012/144> (pages 8, 18).
- [15] Craig Gentry. “Computing arbitrary functions of encrypted data”. In: *Commun. ACM* 53.3 (Mar. 2010), pp. 97–105. ISSN: 0001-0782. DOI: [10.1145/1666420.1666444](https://doi.org/10.1145/1666420.1666444). URL: <https://doi.org/10.1145/1666420.1666444> (page 8).
- [16] Modjtaba Gharibyar, Clemens Krüger, and Dominik Schoop. “Towards Efficient Cloud Data Processing: A Comprehensive Guide to CKKS Parameter Selection”. In: *Proceedings of the 11th International Conference on Information Systems Security and Privacy - Volume 2: ICISSP*. INSTICC. SciTePress, 2025, pp. 502–509. ISBN: 978-989-758-735-1. DOI: [10.5220/0013144800003899](https://doi.org/10.5220/0013144800003899) (page 44).
- [17] Qian Guo et al. “Key Recovery Attacks on Approximate Homomorphic Encryption with Non-Worst-Case Noise Flooding Countermeasures”. In: *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 7447–7461. ISBN: 978-1-939133-44-1. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/guo-qian> (page 42).
- [18] Shai Halevi and Victor Shoup. *Design and implementation of HELib: a homomorphic encryption library*. Cryptology ePrint Archive, Paper 2020/1481. 2020. URL: <https://eprint.iacr.org/2020/1481> (page 35).
- [19] Kristin Lauter. “Private AI: Machine Learning on Encrypted Data”. In: *Recent Advances in Industrial and Applied Mathematics*. Ed. by Tomás Chacón Rebollo, Rosa Donat, and Inmaculada Higuera. Cham: Springer International Publishing, 2022, pp. 97–113. ISBN: 978-3-030-86236-7 (page 2).

- [20] Baiyu Li and Daniele Micciancio. *On the Security of Homomorphic Encryption on Approximate Numbers*. Cryptology ePrint Archive, Paper 2020/1533. 2020. URL: <https://eprint.iacr.org/2020/1533> (page 36).
- [21] Jianwei Li and Phong Q. Nguyen. *A Complete Analysis of the BKZ Lattice Reduction Algorithm*. Cryptology ePrint Archive, Paper 2020/1237. 2020. URL: <https://eprint.iacr.org/2020/1237> (page 34).
- [22] Chiara Marcolla et al. *Survey on Fully Homomorphic Encryption, Theory, and Applications*. Cryptology ePrint Archive, Paper 2022/1602. 2022. DOI: 10.1109/JPROC.2022.3205665. URL: <https://eprint.iacr.org/2022/1602> (pages 7, 11).
- [23] Federico Mazzone et al. “Efficient ranking, order statistics, and sorting under CKKS”. In: *Proceedings of the 34th USENIX Conference on Security Symposium*. USA: USENIX Association, 2025. ISBN: 978-1-939133-52-6 (page 34).
- [24] Daniele Micciancio. *Fully Composable Homomorphic Encryption*. Cryptology ePrint Archive, Paper 2024/1545. 2024. URL: <https://eprint.iacr.org/2024/1545> (page 13).
- [25] Daniele Micciancio and Michael Walter. “On the Bit Security of Cryptographic Primitives”. In: *IACR Cryptol. ePrint Arch.* (2018), p. 77. URL: <http://eprint.iacr.org/2018/077> (page 34).
- [26] Christian Mouchet et al. “Lattigo: a Multiparty Homomorphic Encryption Library in Go”. In: 2020. URL: <https://api.semanticscholar.org/CorpusID:231609747> (page 35).
- [27] OpenMined. *CKKS Explained Series*. <https://openmined.org/blog/ckks-explained-part-1-simple-encoding-and-decoding/> (page 18).
- [28] Oded Regev. “The Learning with Errors Problem”. In: *Proceedings of the 2010 IEEE 25th Annual Conference on Computational Complexity*. CCC '10. USA: IEEE Computer

- Society, 2010, pp. 191–204. ISBN: 9780769540603. DOI: [10.1109/CCC.2010.26](https://doi.org/10.1109/CCC.2010.26). URL: <https://doi.org/10.1109/CCC.2010.26> (page 15).
- [29] Ronald L. Rivest, Michael L. Dertouzos, and Len Adleman. “ON DATA BANKS AND PRIVACY HOMOMORPHISMS”. In: *Foundations of Secure Computation*. New York: Academic Press, 1978. URL: <https://api.semanticscholar.org/CorpusID:6905087> (page 4).
- [30] Sai Sri Sathya et al. *A Review of Homomorphic Encryption Libraries for Secure Computation*. 2018. arXiv: 1812.02428 [cs.CR]. URL: <https://arxiv.org/abs/1812.02428>.
- [31] Aadit Shah, Surindernath Sivakumar, and Prabakaran N. “Encrypted intelligence: A comparative analysis of homomorphic encryption frameworks for privacy-preserving AI”. In: *Journal of Economy and Technology* 4 (2026), pp. 252–265. ISSN: 2949-9488. DOI: <https://doi.org/10.1016/j.ject.2025.08.001>. URL: <https://www.sciencedirect.com/science/article/pii/S2949948825000289> (page 2).
- [32] Jonathan Takeshita et al. *HEProfiler: An In-Depth Profiler of Approximate Homomorphic Encryption Libraries*. Cryptology ePrint Archive, Paper 2024/1059. 2024. URL: <https://eprint.iacr.org/2024/1059> (page 36).
- [33] Weihang Tan et al. “PaReNTT: Low-Latency Parallel Residue Number System and NTT-Based Long Polynomial Modular Multiplication for Homomorphic Encryption”. In: *IEEE Transactions on Information Forensics and Security* 19 (2024), pp. 1646–1659. ISSN: 1556-6021. DOI: [10.1109/tifs.2023.3338553](https://doi.org/10.1109/tifs.2023.3338553). URL: <http://dx.doi.org/10.1109/TIFS.2023.3338553> (page 33).
- [34] William Wolberg et al. *Breast Cancer Wisconsin (Diagnostic)*. UCI Machine Learning Repository. 1993. URL: <https://doi.org/10.24432/C5DW2B> (page 71).

- [35] Nuo Xu et al. *FHE-Agent: Automating CKKS Configuration for Practical Encrypted Inference via an LLM-Guided Agentic Framework*. 2025. arXiv: 2511.18653 [cs.CR]. URL: <https://arxiv.org/abs/2511.18653> (pages 3, 41).
- [36] Dikai Zhao. “Advances and Applications in Fully Homomorphic Encryption Research”. In: *Applied and Computational Engineering* 135 (Feb. 2025), pp. 39–48. DOI: 10.54254/2755-2721/2025.20959 (page 10).

APPENDIX A

GENERATIVE AI USAGE STATEMENT

Generative AI was used in a limited and supportive manner throughout this project. All core ideas, experimental design, and conclusions presented in this thesis are my original work, and AI tools were not used to generate substantive content.

For the writing of this thesis, generative AI was used for the high-level brainstorming of information to include, particularly for the material in Chapter 3 and Chapter 6. It was also used for overarching feedback regarding comprehensibility, especially for Chapter 1, Chapter 6, and Chapter 7.

Generative AI was used for formatting in $\text{\LaTeX}$, including refining figure and table environments. Additionally, generative AI was used for sentence-level revisions of my own writing to improve clarity and readability. The tools were not used for generating sentences or full paragraphs in this work.

The following sentences were revised for clarity with generative AI:

- Therefore, directly sharing raw data with external services is often impractical or prohibited, limiting the adoption of powerful machine learning tools in privacy-critical domains (Chapter 1).
- In particular, these candidate configurations are evaluated using a proposed representative test vector strategy to approximate inference error without requiring direct use of sensitive data (Chapter 1).

- This chapter provides an overview of the current state of CKKS parameter selection, highlighting the provided parameter functionality of CKKS libraries, as well as the current automation and simplification research efforts and tools (Chapter 3).
- As a result, there remains a gap in tools that allow developers to systematically explore parameter configurations that reflect application-specific requirements, particularly in practical cloud-based machine learning scenarios (Chapter 3).
- The experiments in this chapter assess whether the proposed tool can efficiently and effectively explore the CKKS parameter space and provide accurate recommendations for practical machine learning inference tasks (Chapter 5).
- This distinction is particularly important in privacy-sensitive settings, where a central objective is to minimize data exposure (Chapter 5).
- Furthermore, generating representative vectors is computationally more viable as the process does not scale with dataset size, making it a more practical approach for large datasets (Chapter 5).
- The results of this work suggest that parameter selection can instead be grounded in the structure of the underlying dataset-model workload (Chapter 6).
- By providing a structured view of the trade-off space, the tool supports more transparent and systematic decision-making, making homomorphic encryption more accessible to developers and organizations (Chapter 6).
- Privacy concerns will continue to shape the future of machine learning and artificial intelligence, increasing the need for tools that make advanced cryptographic techniques more accessible (Chapter 7).

For this project, generative AI was used as an aid during the development of software components. Specifically, it was used to brainstorm implementation approaches, assist with debugging errors, and help with the incorporation of external libraries.

